

Bilgisayar Mühendisliği Bölümü
BIMU2057 Veri Yapıları

Dr. Öğr. Üyesi Nihal ALTUNTAŞ

nihal.altuntas@iuc.edu.tr

Bu Derste

- Liste
- Bağlı Listeler (Linked List)
- Bağlı Liste Çeşitleri

Listeler

- Aynı tip elemanlardan oluşan veri yapısı
- Listenin boyu değişkendir
- Başlıca işlemler
  - ListeyiYaz
  - Ekle
  - Sil

Listeler

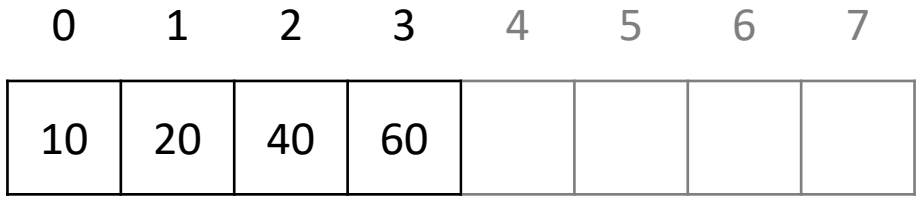
- Peki dizi kullansak?

- ListeYaz ya da Bul işlemleri lineer işlemlerdir.
  - Liste elemanları baştan sona sırayla taranır.

- Ekle ya da Sil işlemleri, dizi üzerinde yapılması yavaş işlemlerdir.
  - Araya eklemek için, listede eklenen elemanın sağında kalan tüm elemanlar 1 sağa kaydırılmalı.
  - Eleman silindiğinde sağdaki elemanlar 1 sola kaydırılarak boşluk kapatılmalı.

Listeler

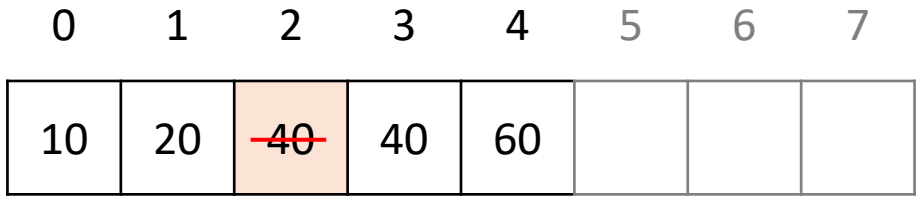
- Elimizde bir dizi (`list`) olsun



- Bu diziye sıralamayı bozmadan 30 eklemek istiyoruz.
  - Ne yapmak gerekir?

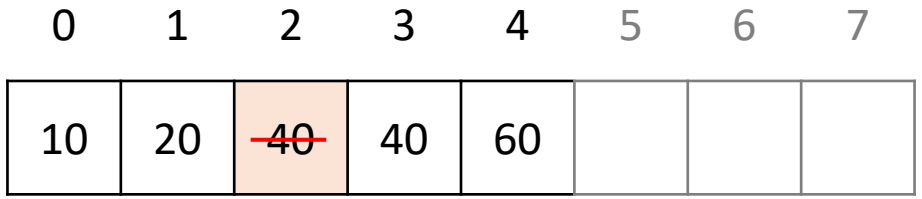
Listeler

- Adım 1: 30’dan büyük sayıları 1 sağa kaydır

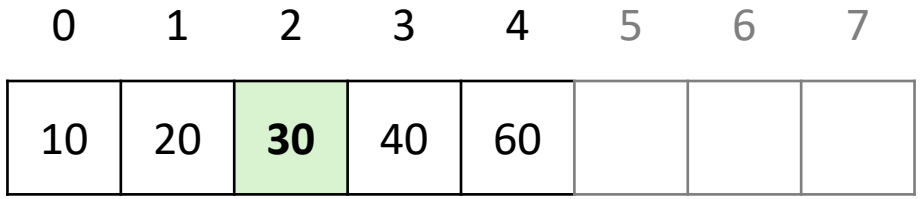


Listeler

- Adım 1: 30’dan büyük sayıları 1 sağa kaydır

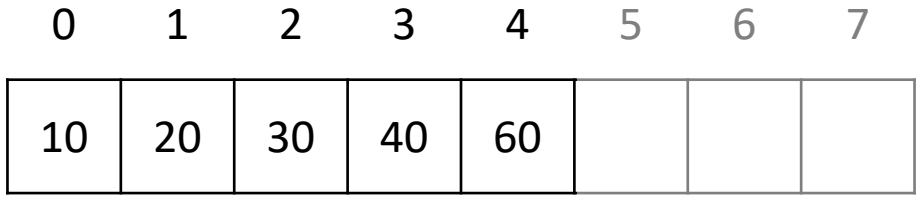


- Adım 2: 30 değerini ilgili indekse kopyala



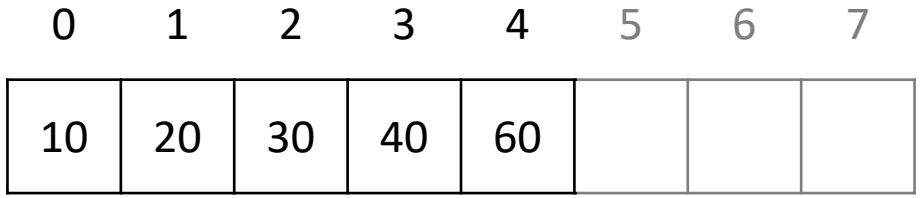
Listeler

- Peki listeden 20’yi silmek istersek?

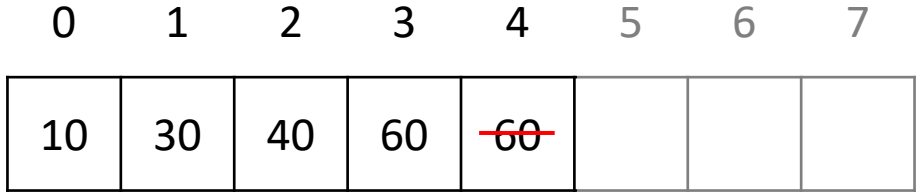


Listeler

- Peki listeden 20'yi silmek istersek?

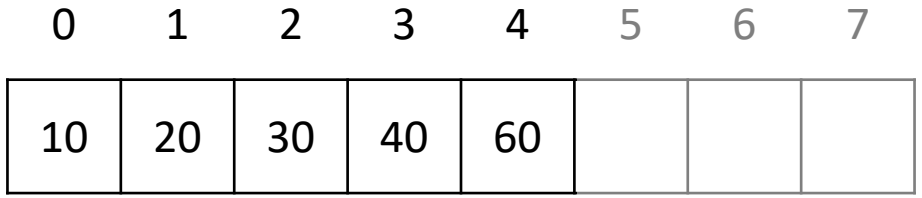


  - Adım 1: 20 sayısının sağındaki değerleri 1 sola kaydır

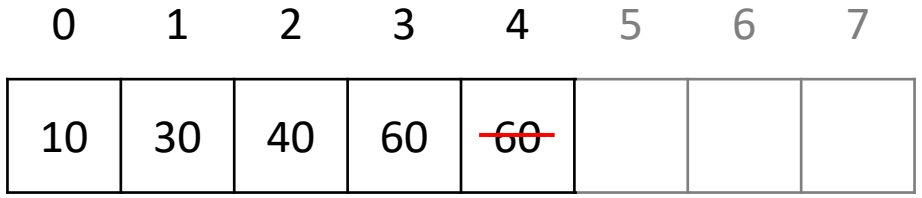


Listeler

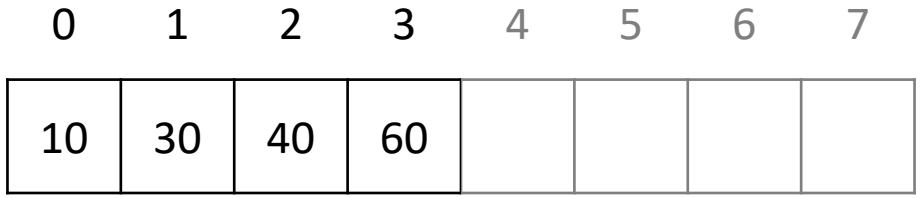
- Peki listeden 20'yi silmek istersek?



  - Adım 1: 20 sayısının sağındaki değerleri 1 sola kaydır

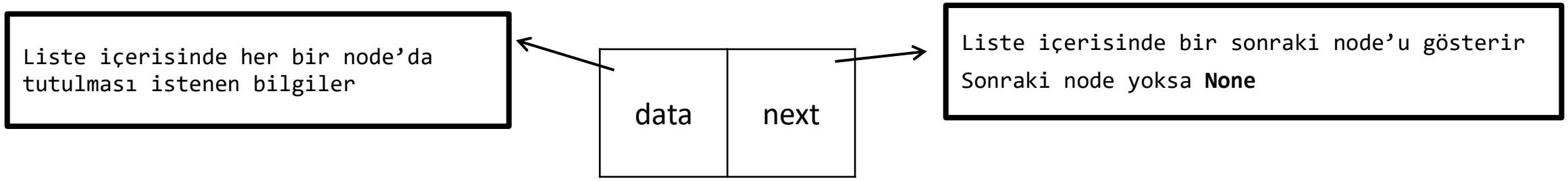


  - Adım 2: En sonda oluşan mükerrer durumu kaldır



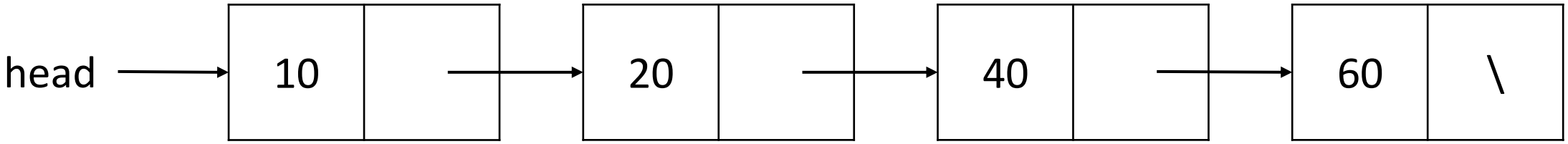
Bağlı Listeler (Linked List)

- Birbirlerine bağlanmış “veri”leri içeren yapılarıdır
- Her bir “veri” düğüm (node) olarak isimlendirilir
- Node’lar genellikle farklı alanlardan oluşan `sınıf` yapısındadır



Bağlı Listeler (Linked List)

- Bağlı listenin düğümleri, arraylerden farklı olarak, bellekte birbirini izleyen alanlarda yer almazlar.
- Bir listeye erişmek için, ilk node bir değişkenle (head) tutulur.
  - Listeyi baştan sona dolaşmak için, node'dan node'a next alanı üzerinden geçilir.
  - Böylece zincirleme olarak tüm node'lara tek tek erişilmiş olur.



Linked List Başlıca İşlemler

- Listeyi Yaz
- Yeni Node Ekle
  - Sona ekle
  - Başa ekle
  - İndekse ekle
  - Sıralı ekle
- Node Sil
  - Sondan sil
  - Baştan sil
  - İndeksten sil
  - Değere göre sil

Linked List Sınıf Yapısı

```
class LinkedList:
   def __init__(self):
      self.head = None

   def printList(self):

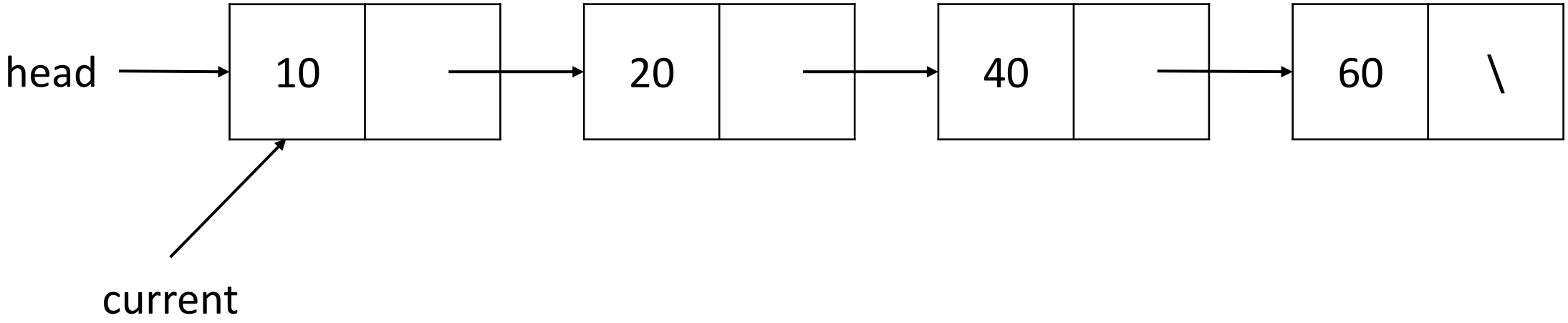
   def insertLast(self, node):
   def insertFirst(self, node):
   def insertAtIndex(self, idx, node):
   def insertSorted(self, node):

   def removeLast(self):
   def removeFirst(self):
   def removeAtIndex(self, idx):
   def removeValue(self, value):
```

```
class Node:
   def __init__(self, newData):
      self.data = newData
      self.next = None
```

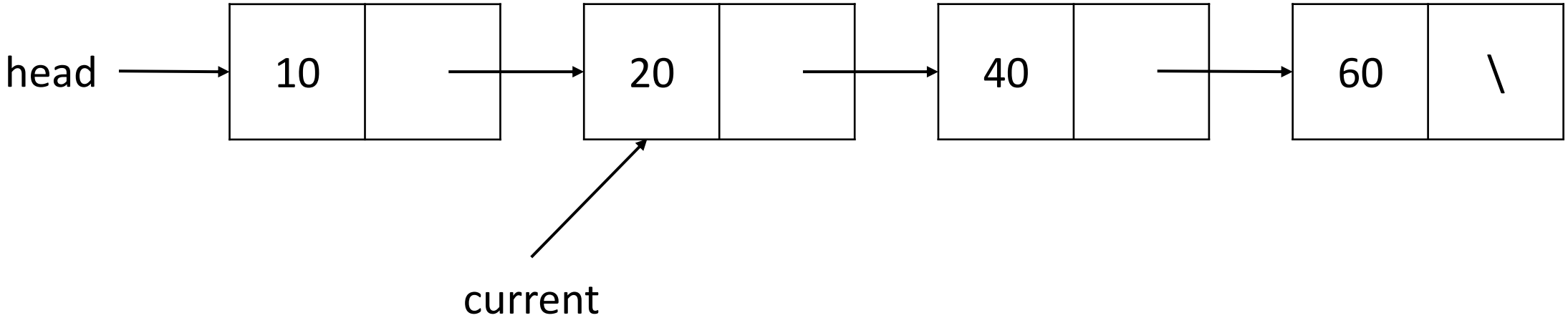
Linked List

- Listeyi Yaz



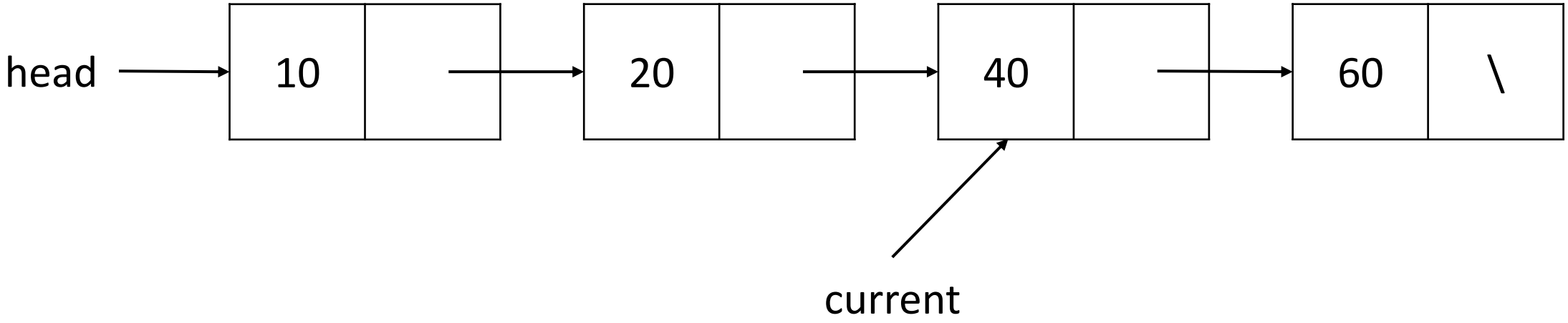
Linked List

- Listeyi Yaz



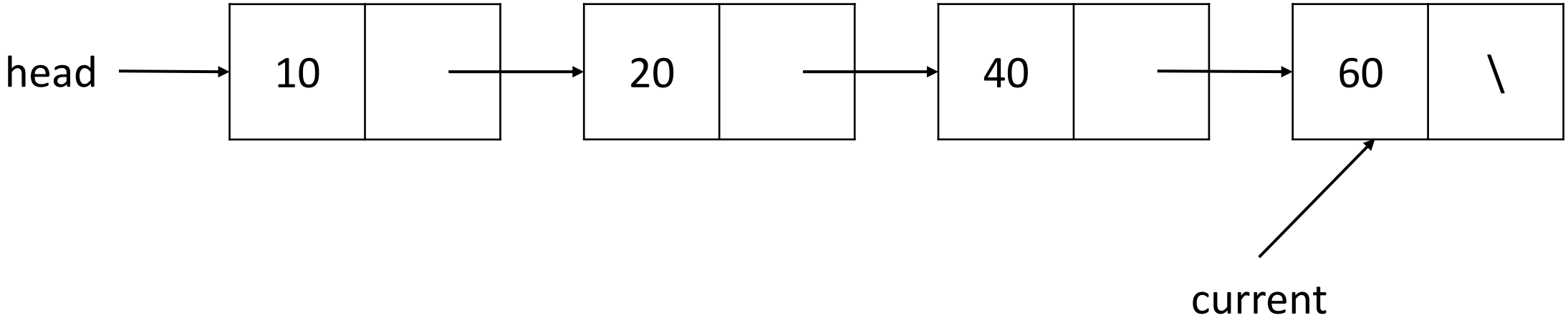
Linked List

- Listeyi Yaz



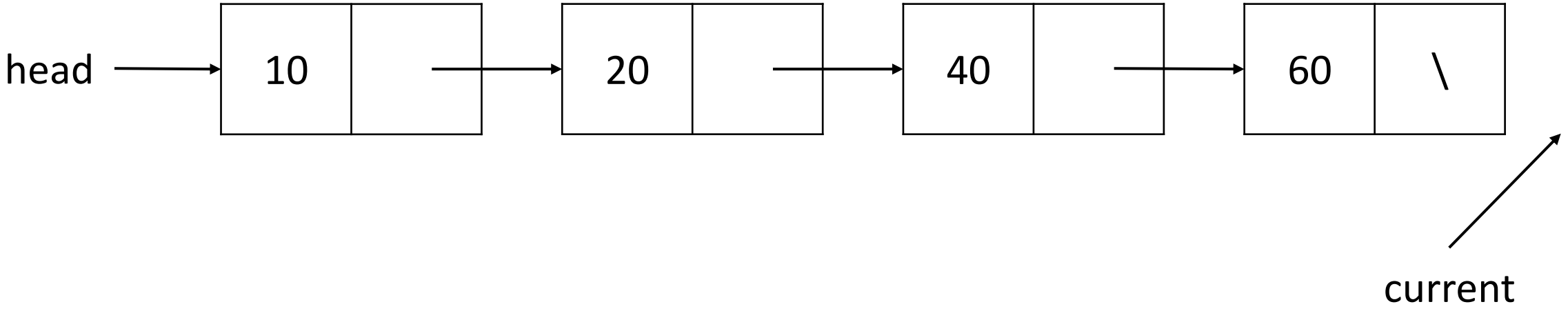
Linked List

- Listeyi Yaz



Linked List

- Listeyi Yaz



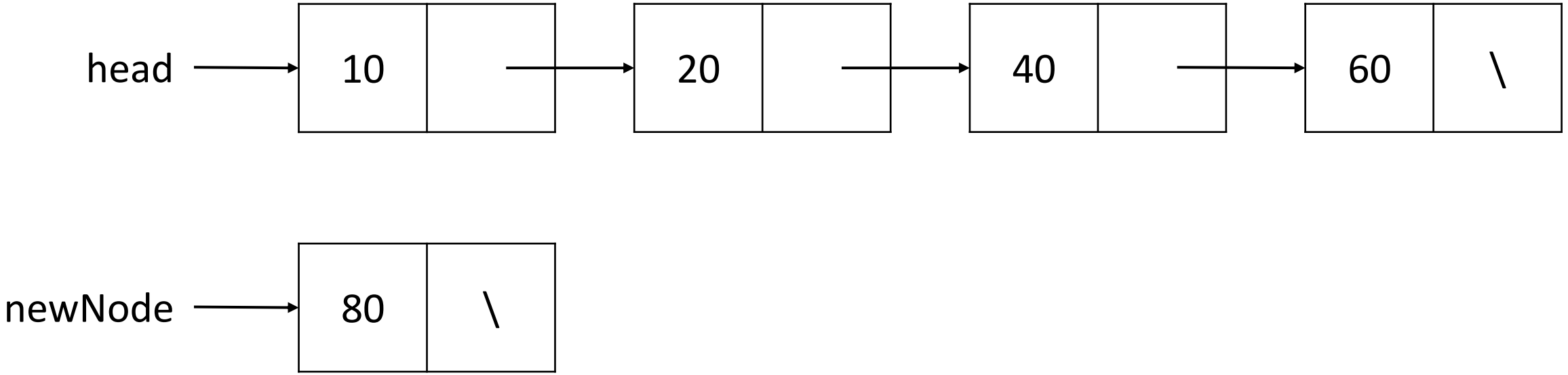
Linked List

- Listeyi Yaz Pseudocode

```
printList()
    current = head
    while(current != None)
        print current
        current = current.next
```

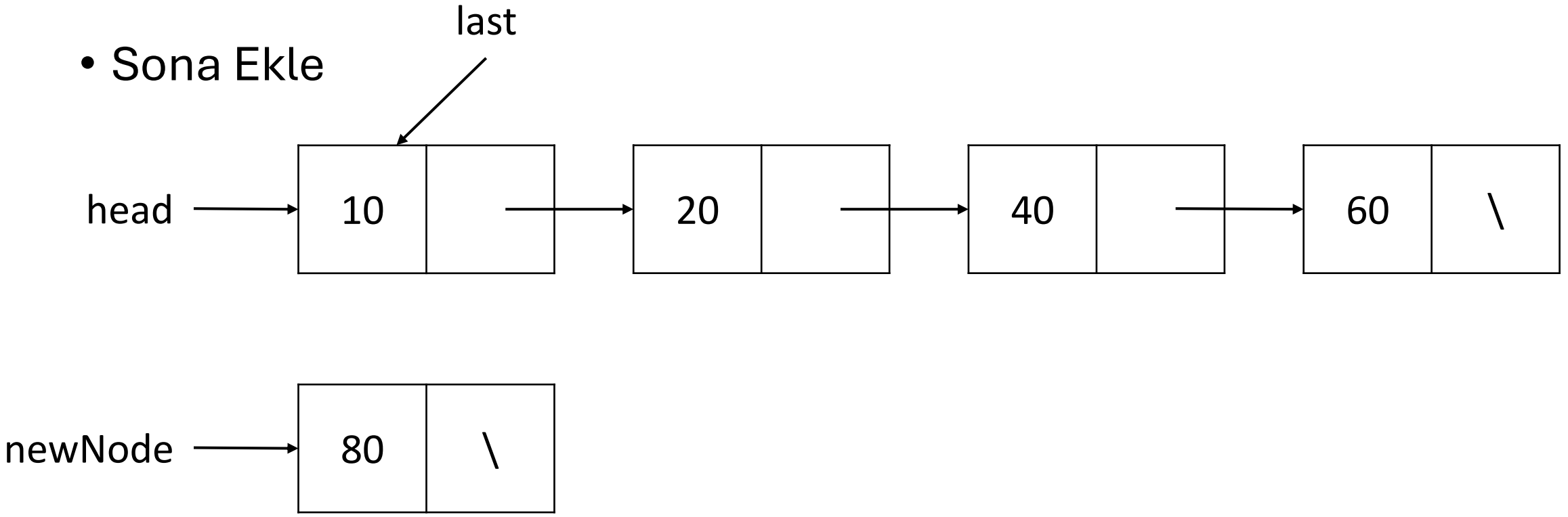
Linked List

- Sona Ekle



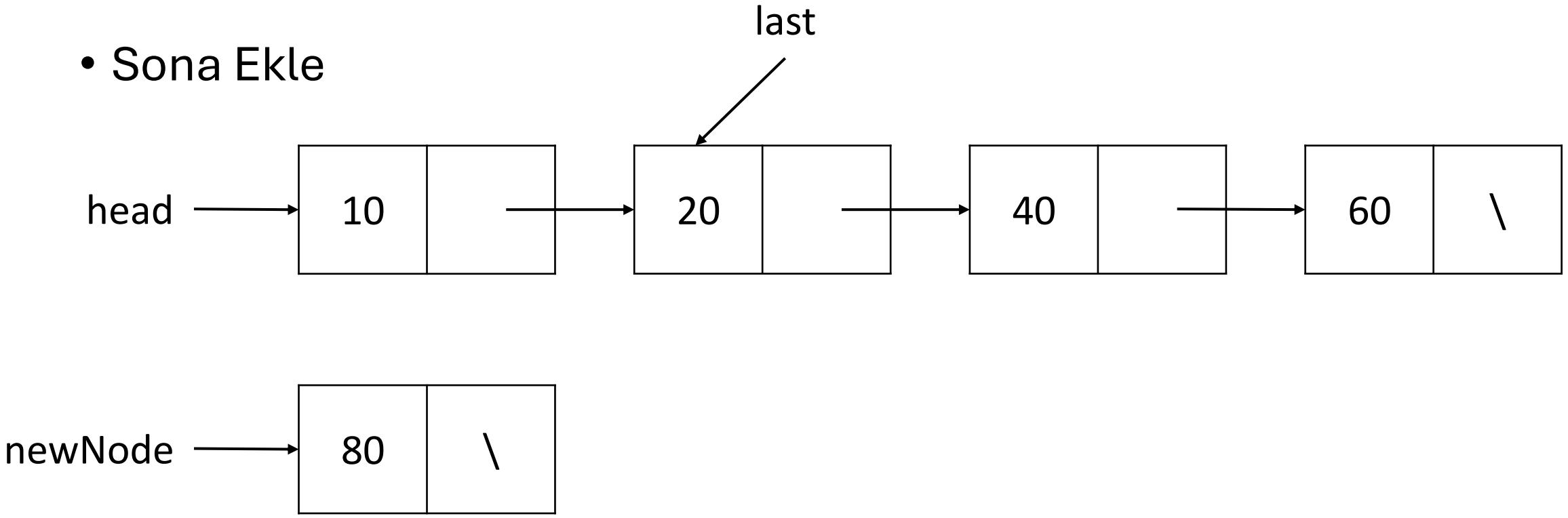
Linked List

- Sona Ekle



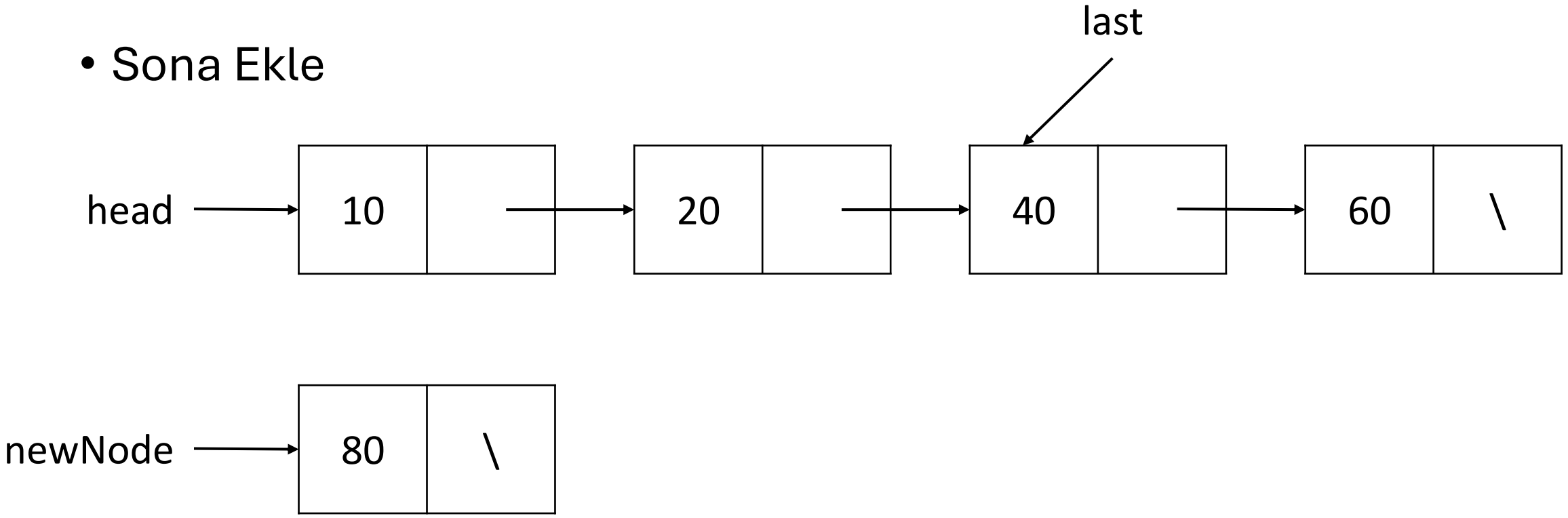
Linked List

- Sona Ekle



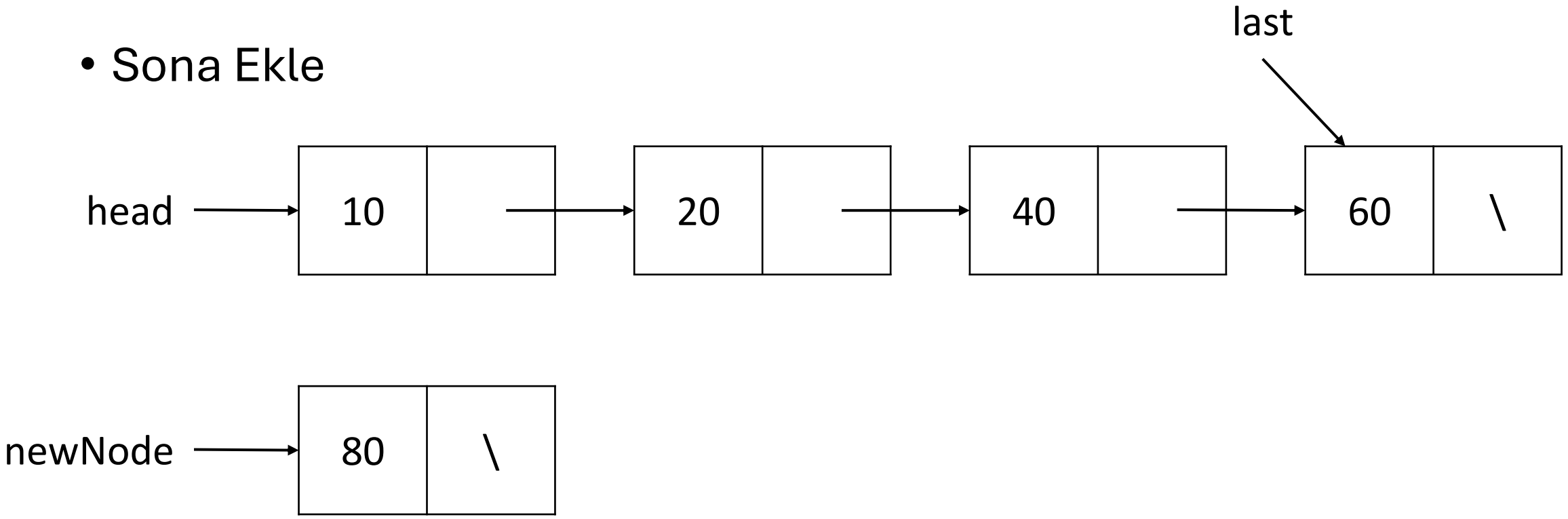
Linked List

- Sona Ekle



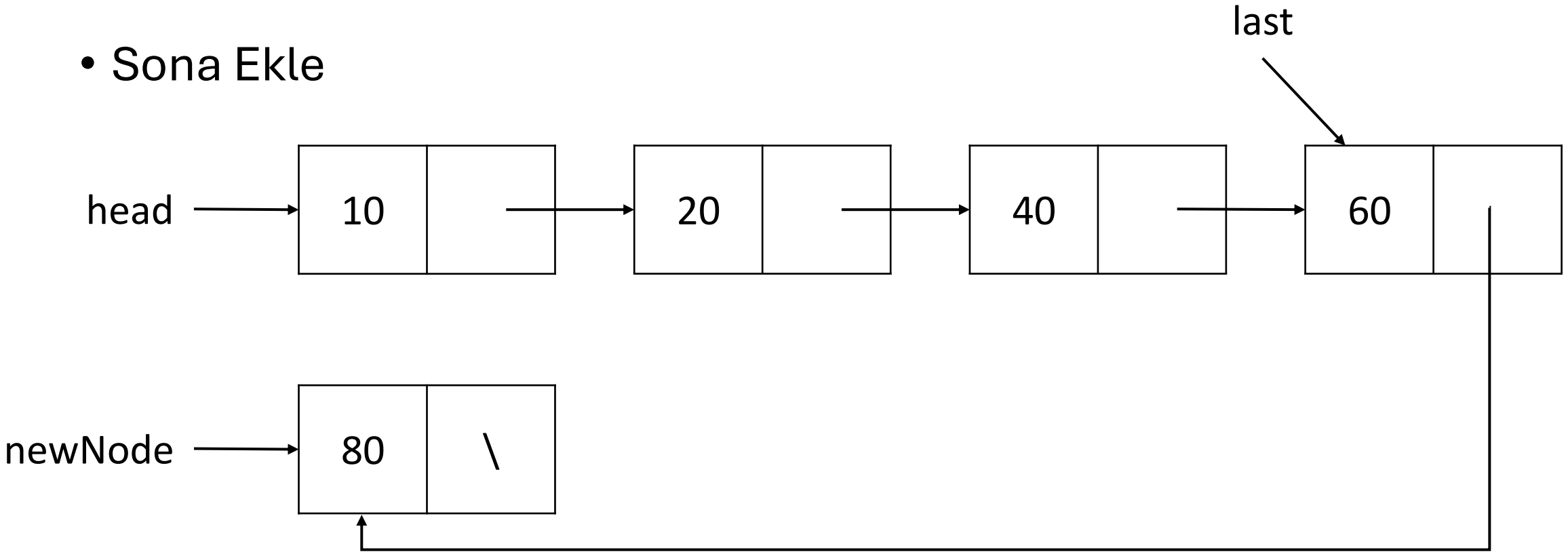
Linked List

- Sona Ekle



Linked List

- Sona Ekle



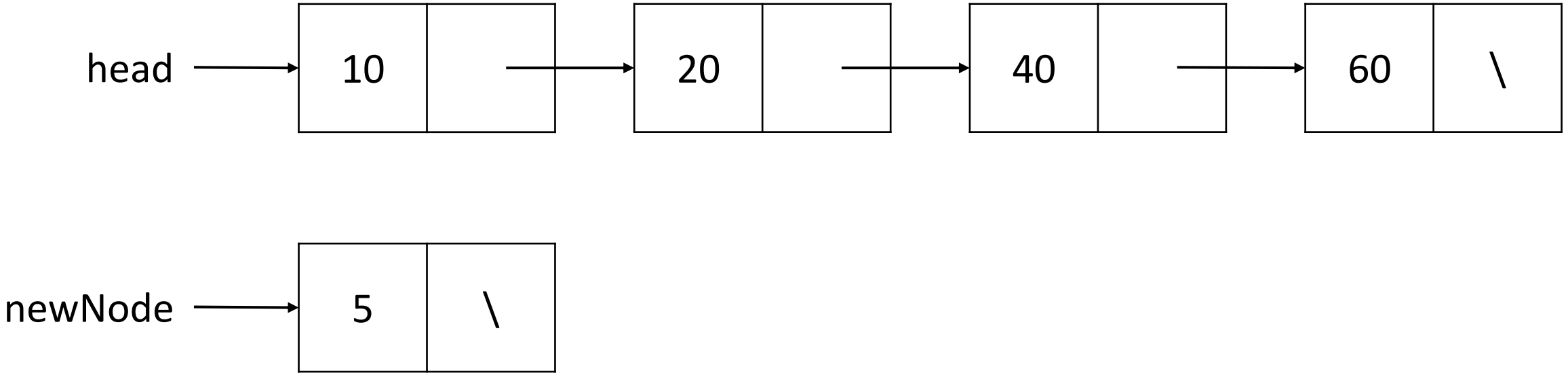
Linked List

- Sona Ekle Pseudocode

```
insertLast(newNode)
    if(newNode != None)
        if(head == None)
            head = newNode
        else
            last = head
            while(last.next != None)
                last = last.next
            last.next = newNode
```

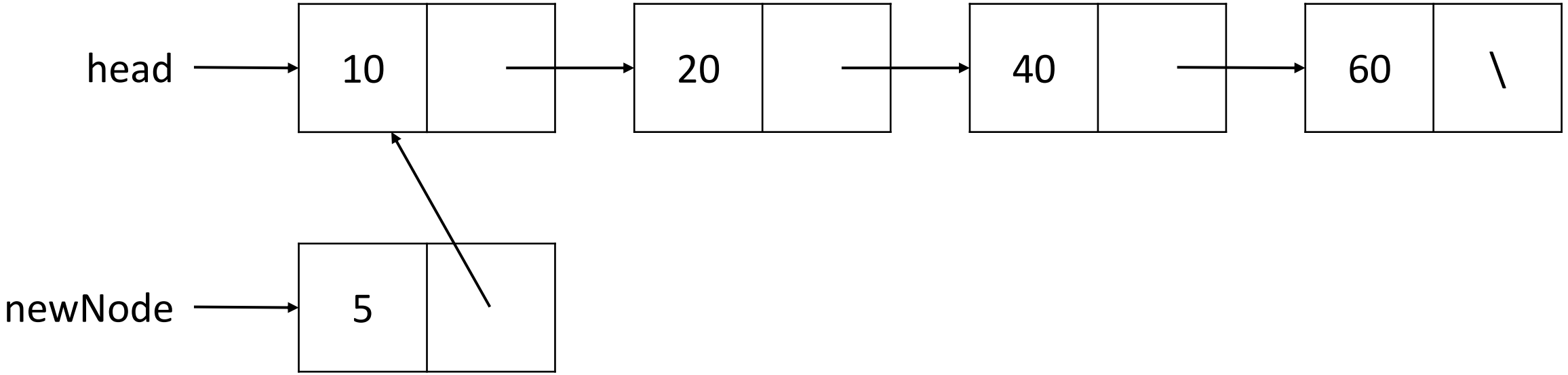
Linked List

- Başa Ekle



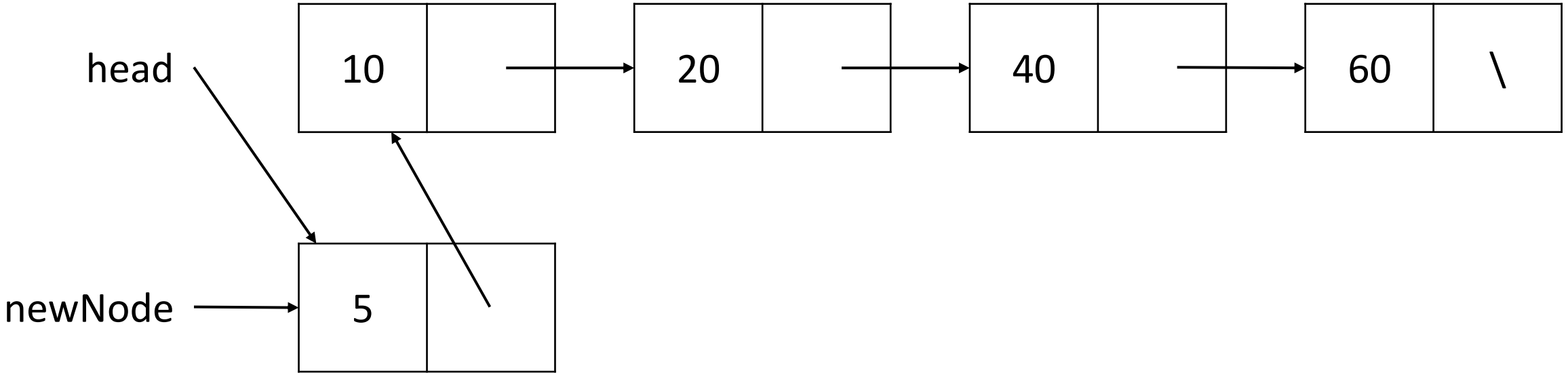
Linked List

- Başa Ekle



Linked List

- Başa Ekle



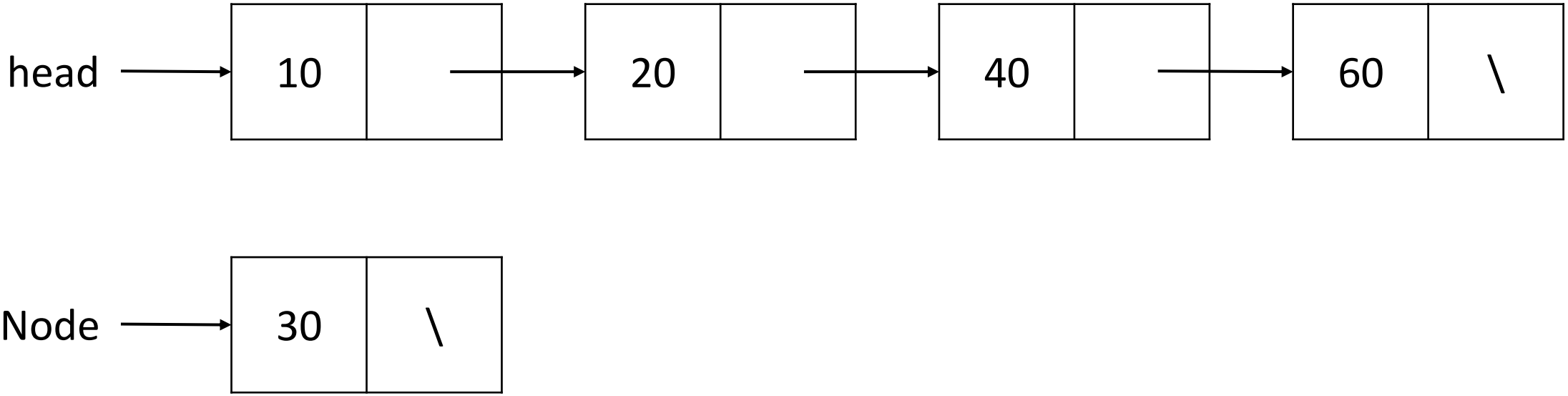
Linked List

- Başa Ekle Pseudocode

```
insertFirst(newNode)
    if(newNode != None)
        newNode.next =head
        head.next = newNode
```

Linked List

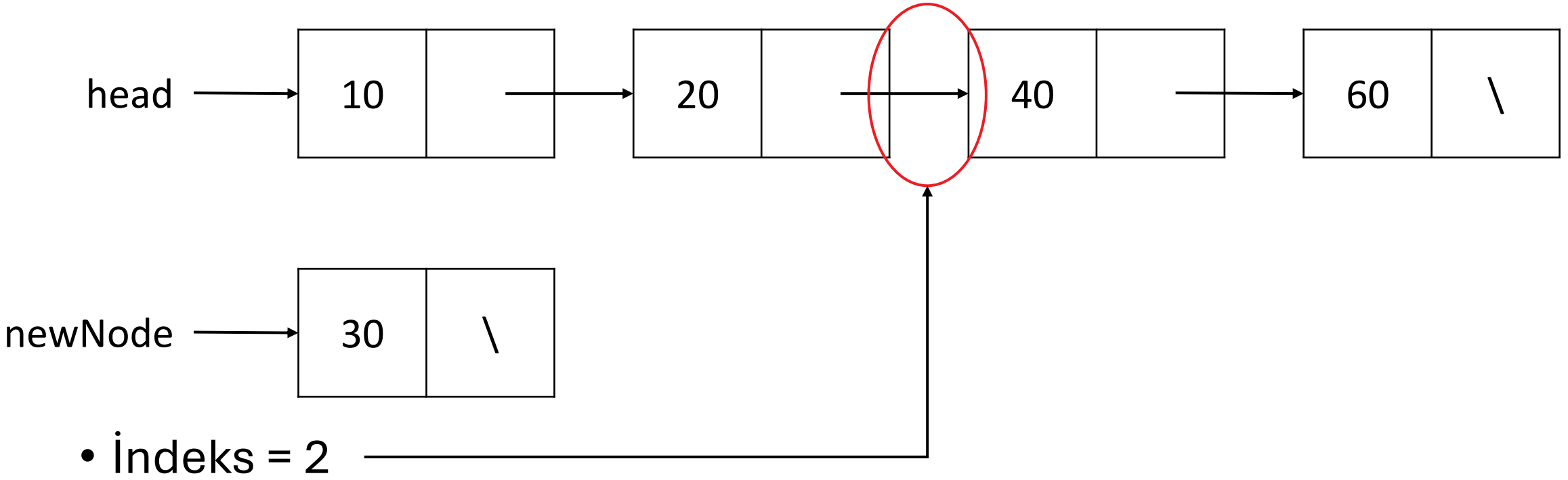
- İndekse Ekle



- İndeks = 2

Linked List

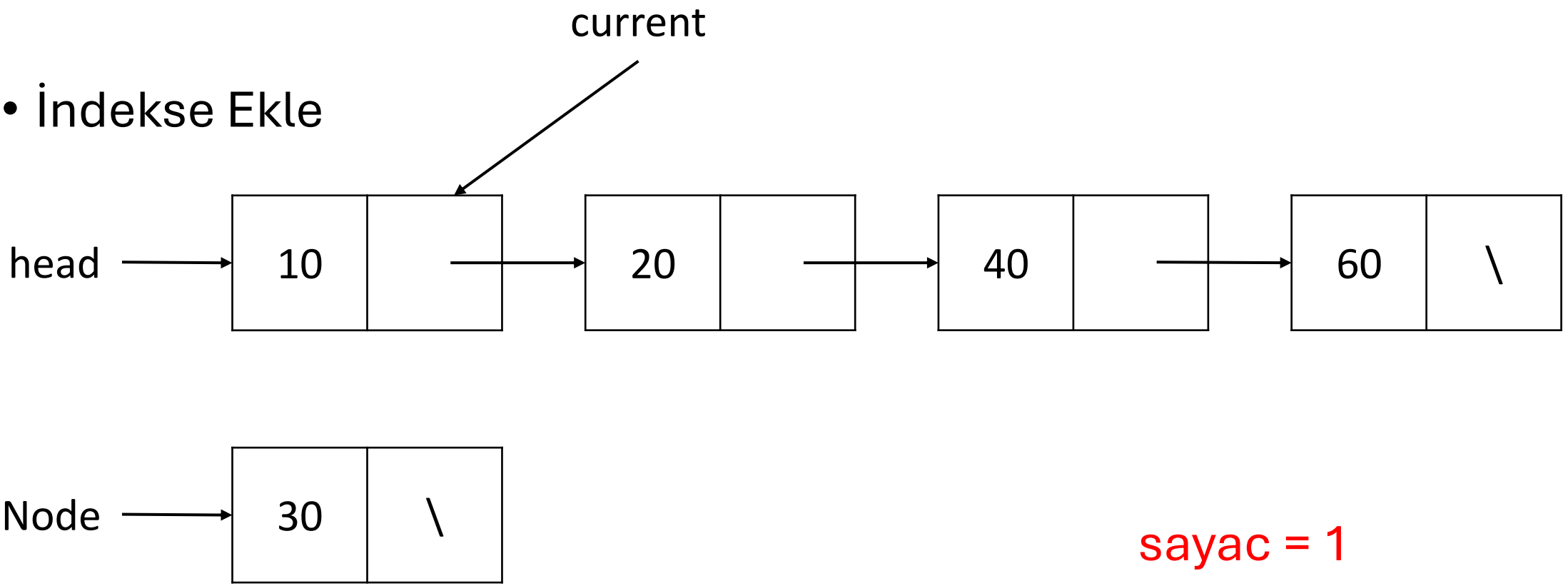
- İndekse Ekle



- İndeks = 2

Linked List

- İndekse Ekle

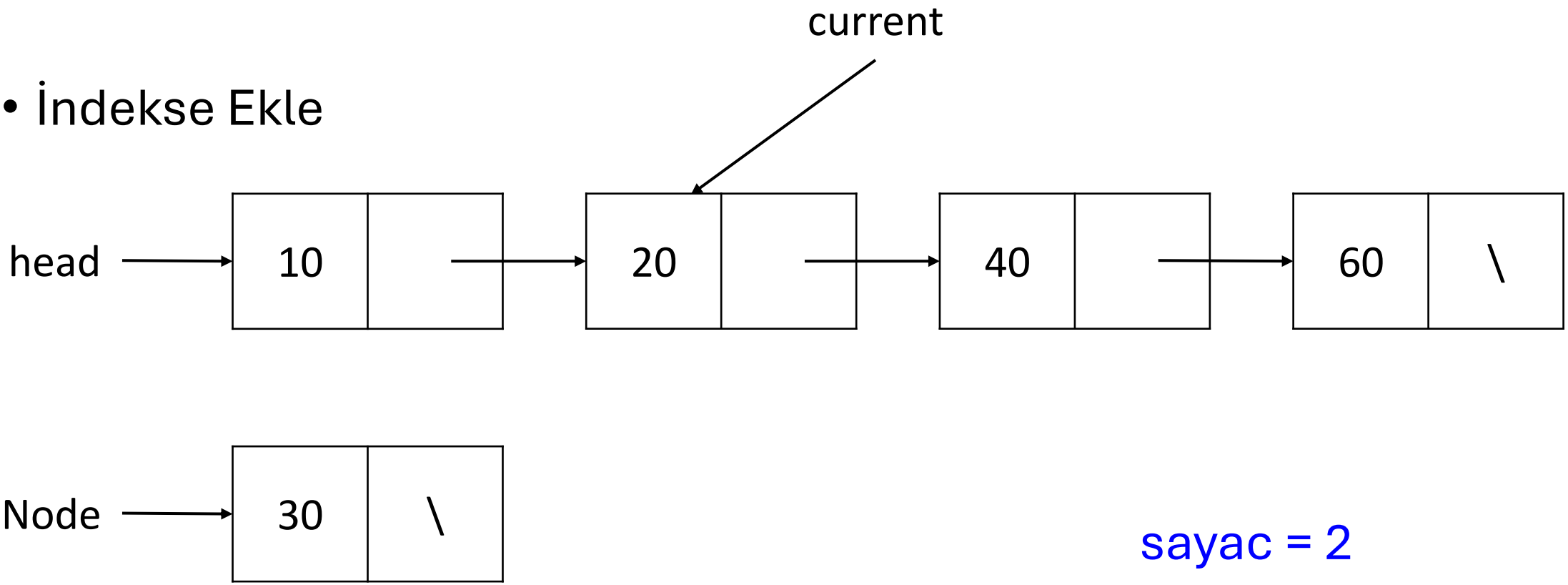


sayac = 1

- İndeks = 2

Linked List

- İndekse Ekle

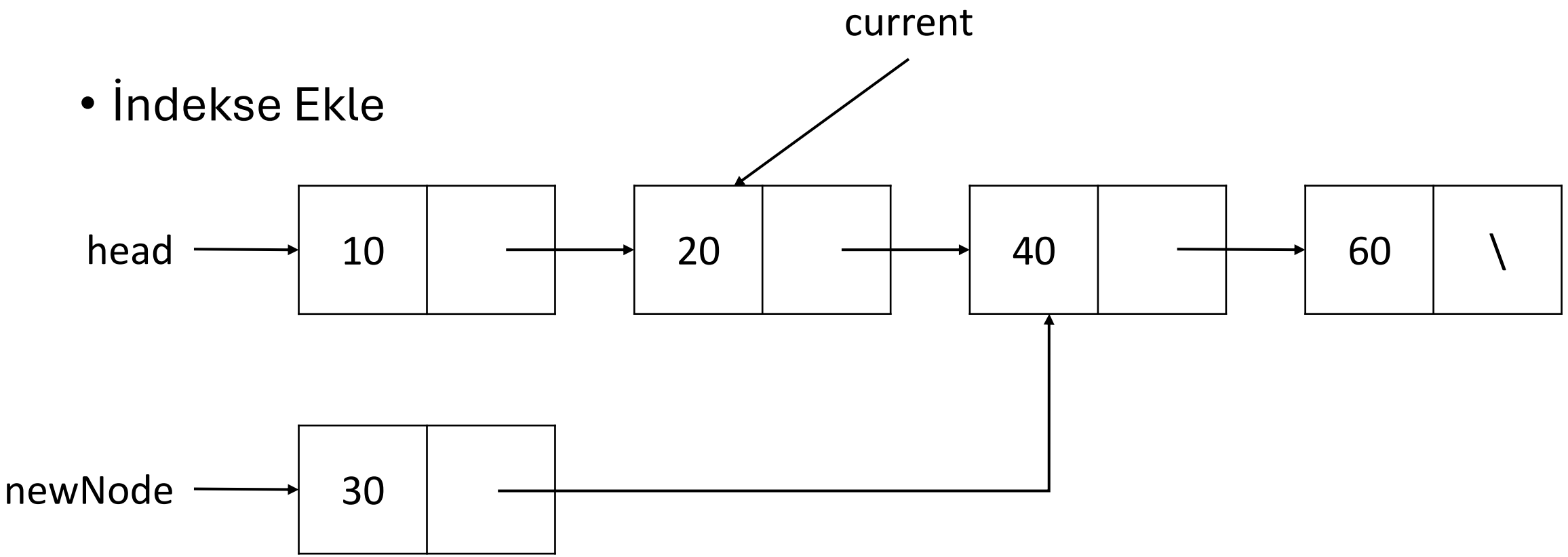


sayac = 2

- İndeks = 2

Linked List

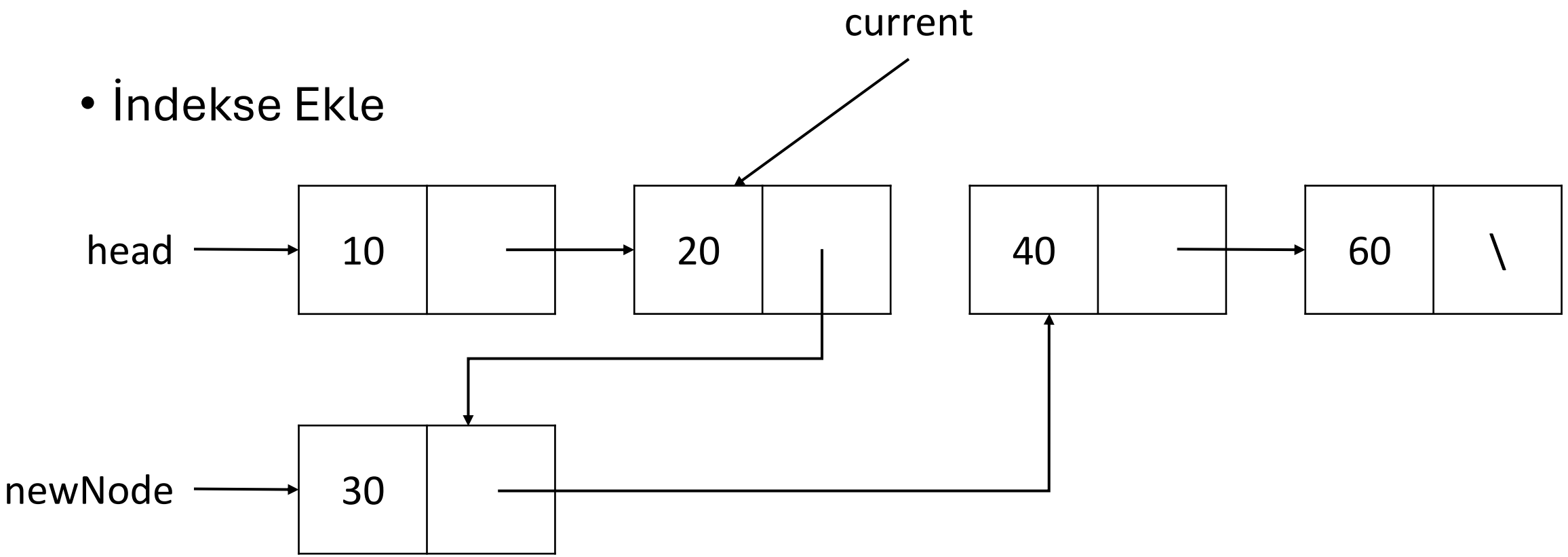
- İndekse Ekle



- İndeks = 2

Linked List

- İndekse Ekle



- İndeks = 2

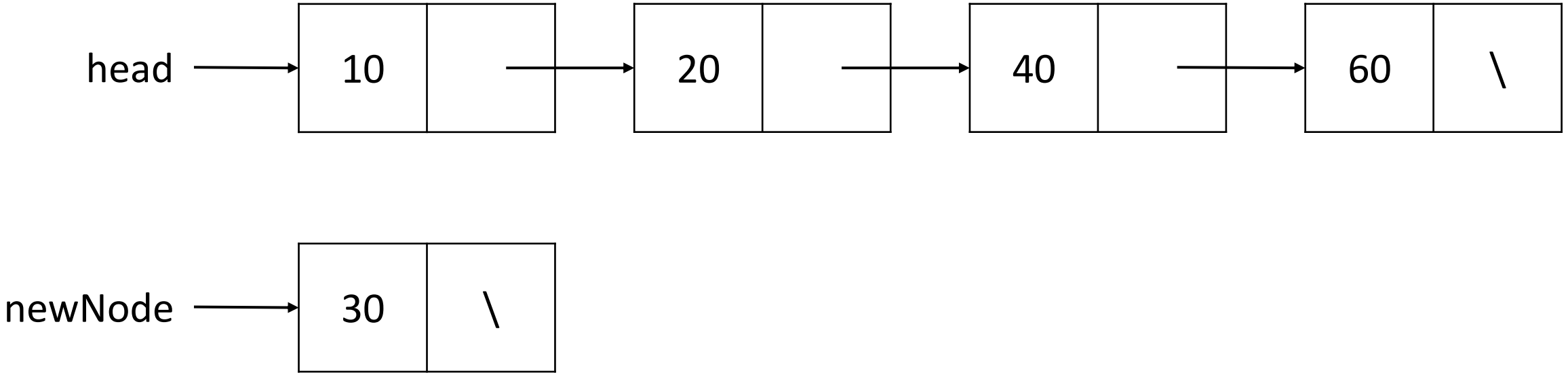
Linked List

- İndekse Ekle Pseudocode

```
insertAtIndex(idx, newNode):
    if idx <= 0
        insertFirst(newNode)
    current = head
    for (idx-1) times
        if current.next != None
            current = current.next
    newNode.next = current.next
    current.next = newNode
```

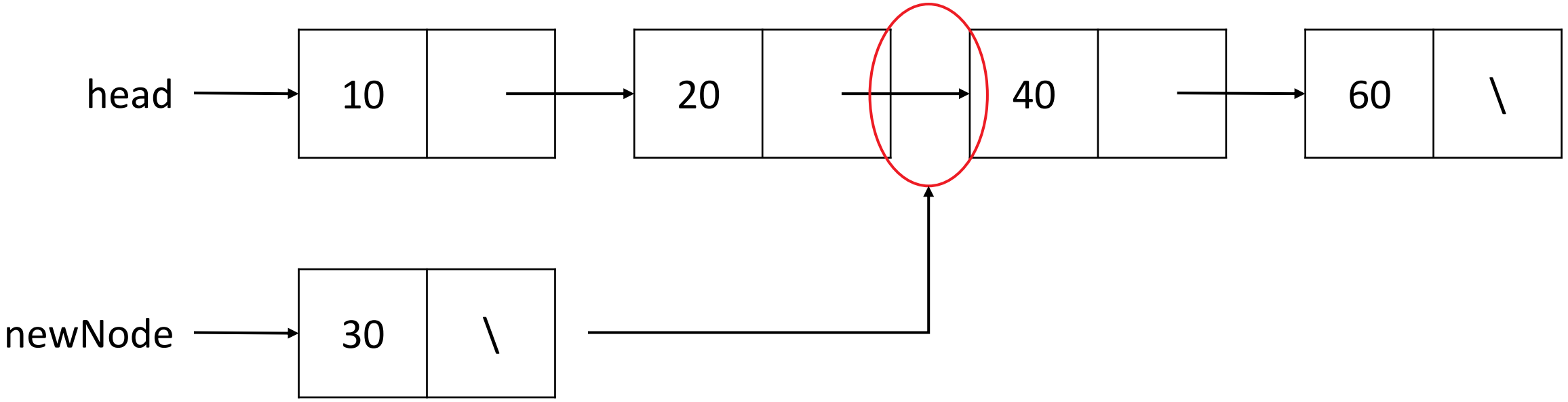
Linked List

- Sıralı Ekle



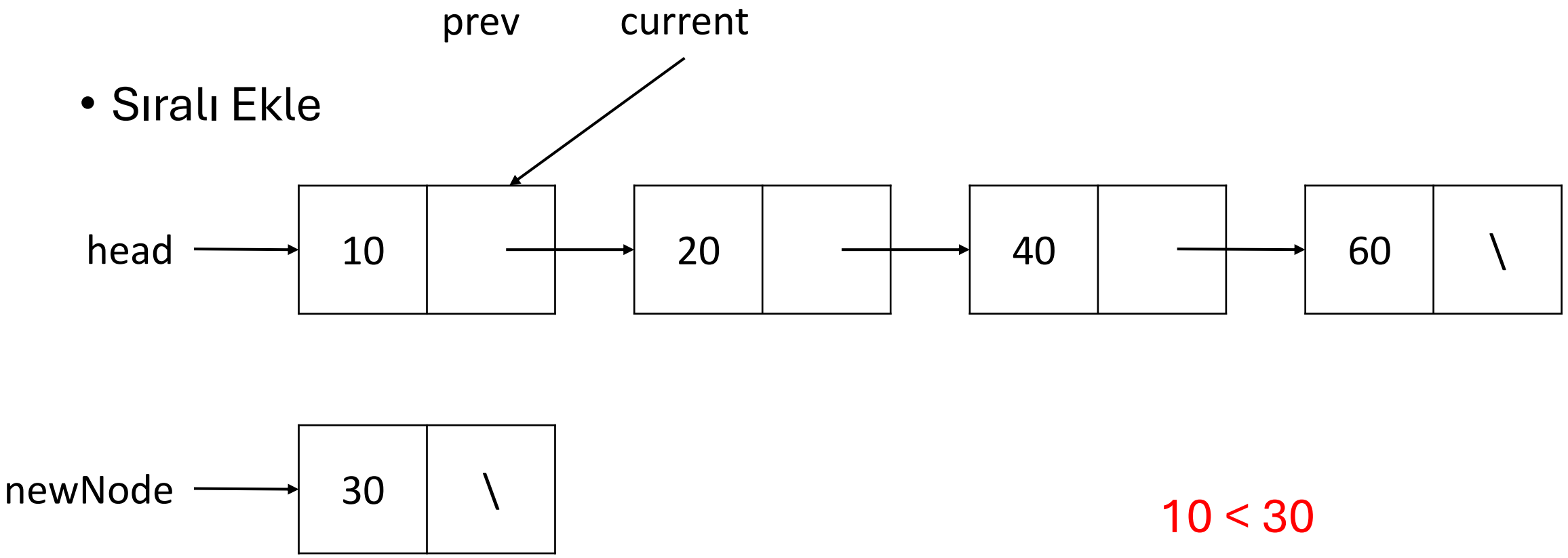
Linked List

- Sıralı Ekle



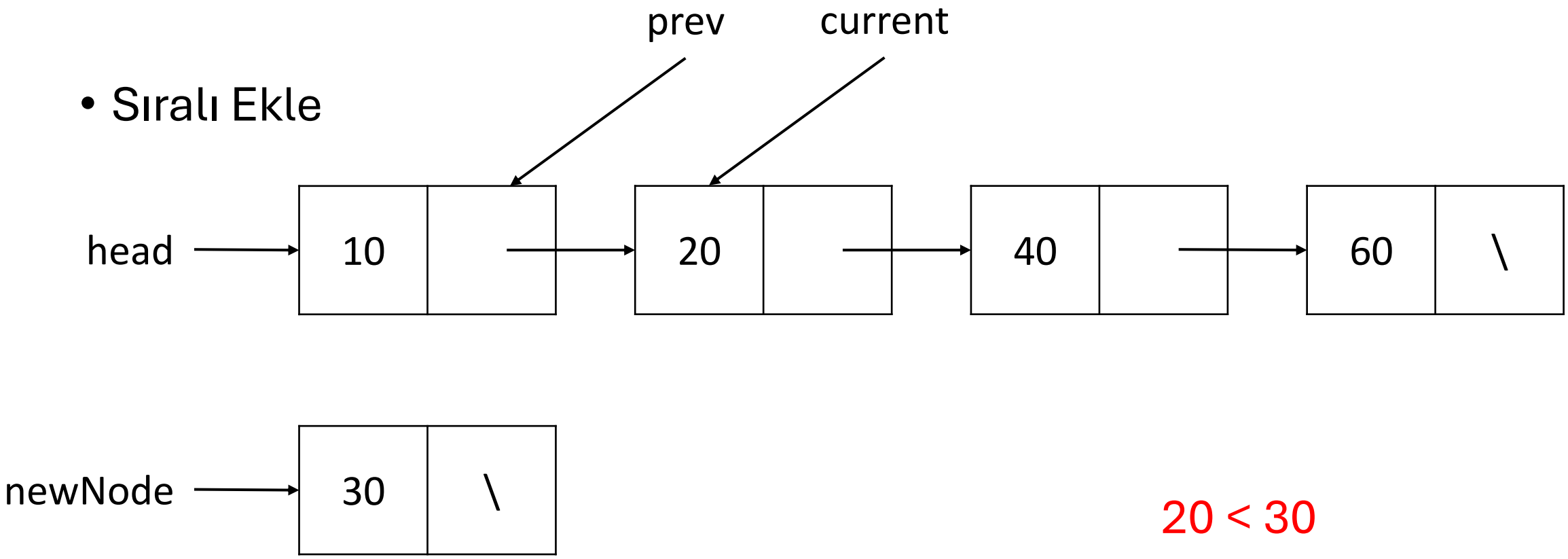
Linked List

- Sıralı Ekle



Linked List

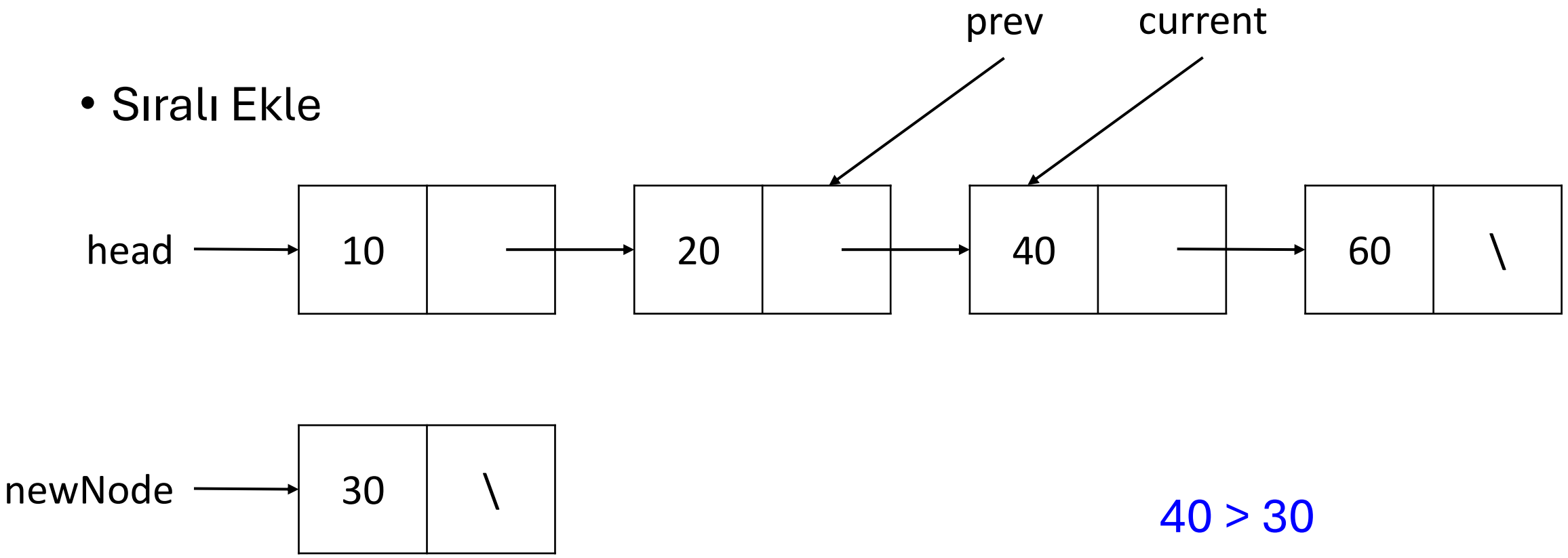
- Sıralı Ekle



20 < 30

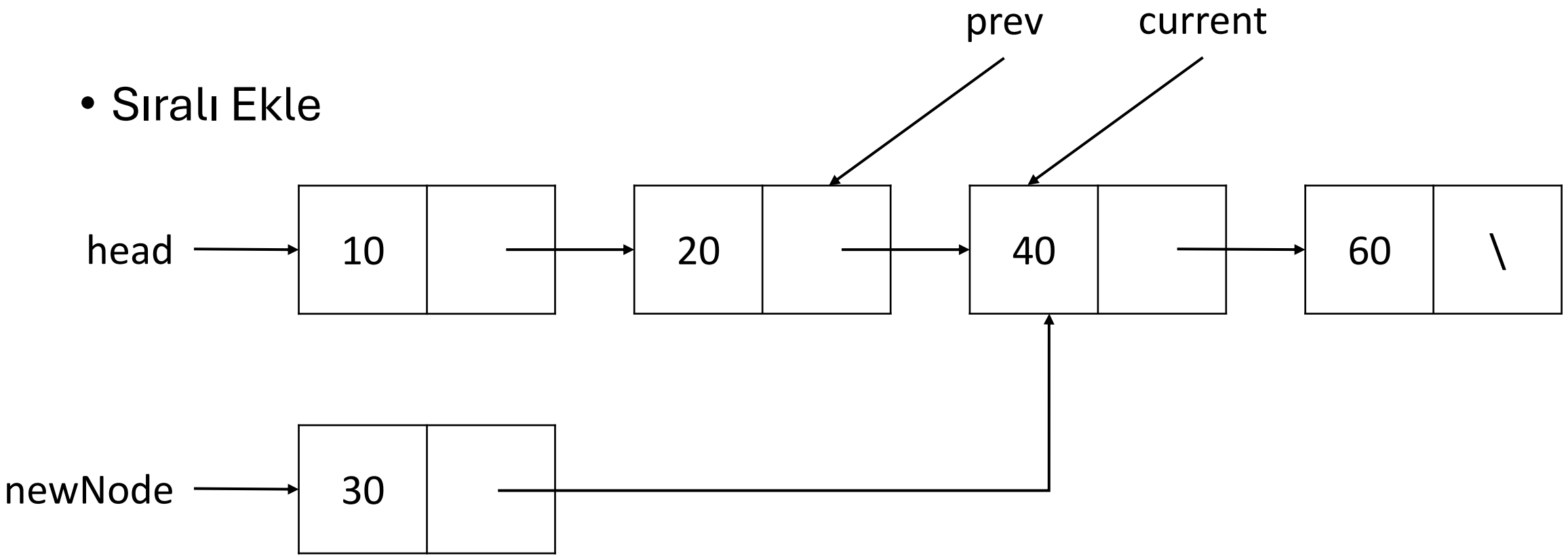
Linked List

- Sıralı Ekle



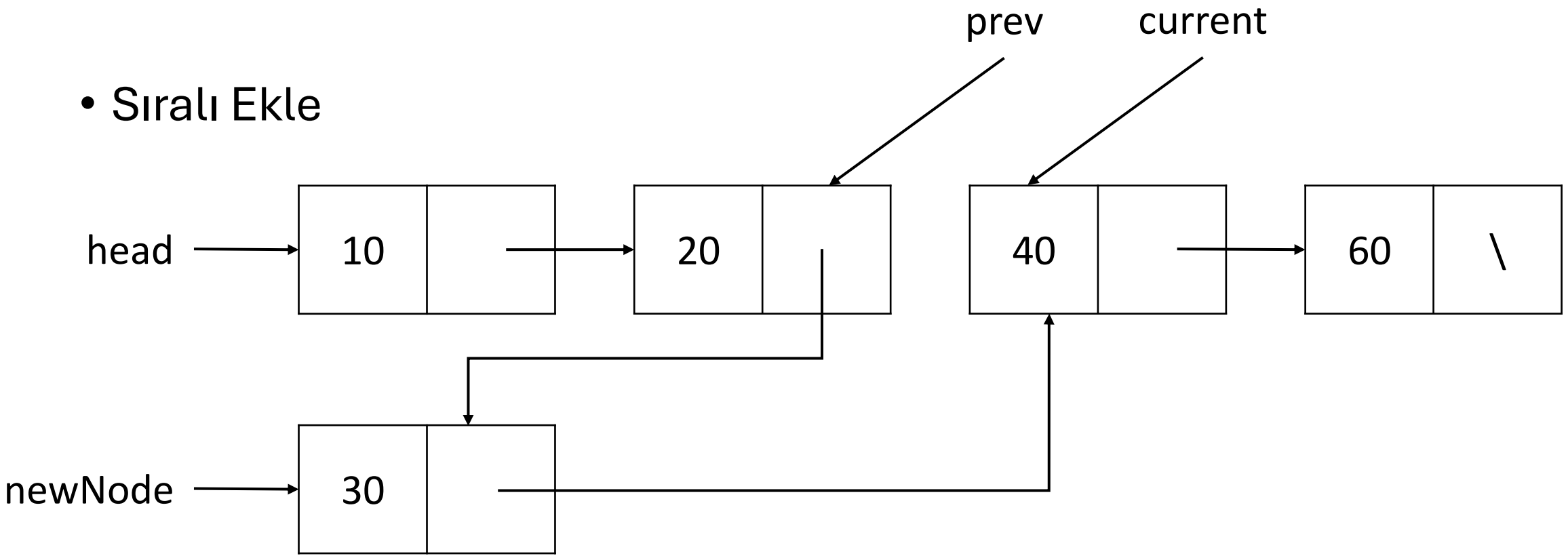
Linked List

- Sıralı Ekle



Linked List

- Sıralı Ekle



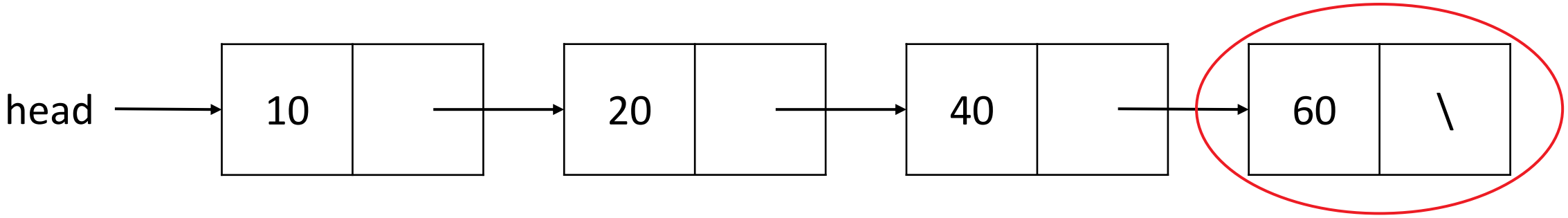
Linked List

- Sıralı Ekle Pseudocode

```
insertSorted(newNode)
    if newNode.data < head.data
        insertFirst(newNode)
    else
        current = head
        while current != None AND newNode.data > current.data
            prev = current
            current = current.next
        newNode.next = current
        prev.next = newNode
```

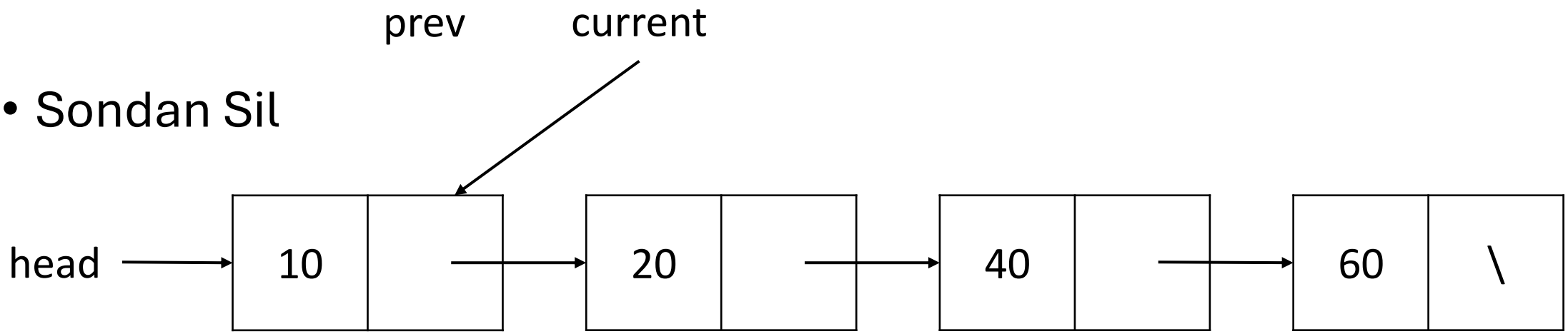
Linked List

- Sondan Sil



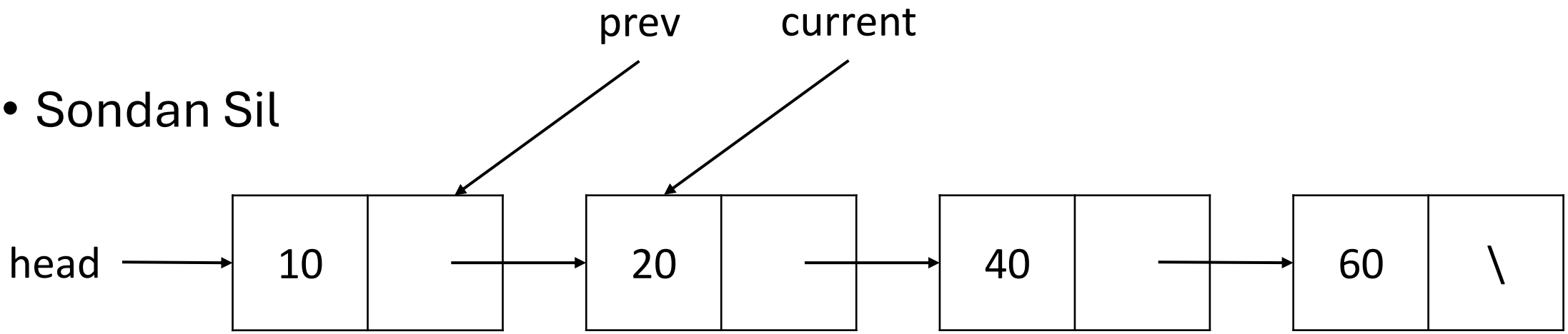
Linked List

- Sondan Sil



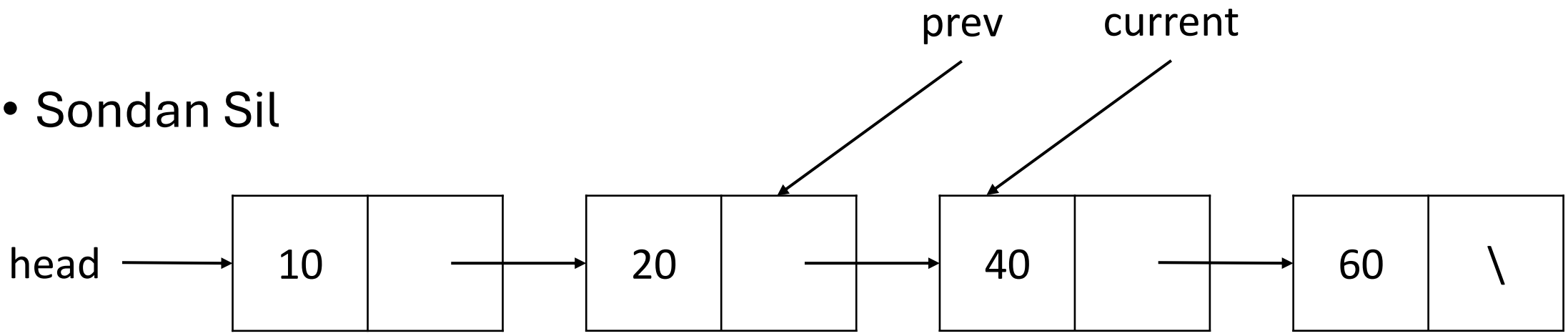
Linked List

- Sondan Sil



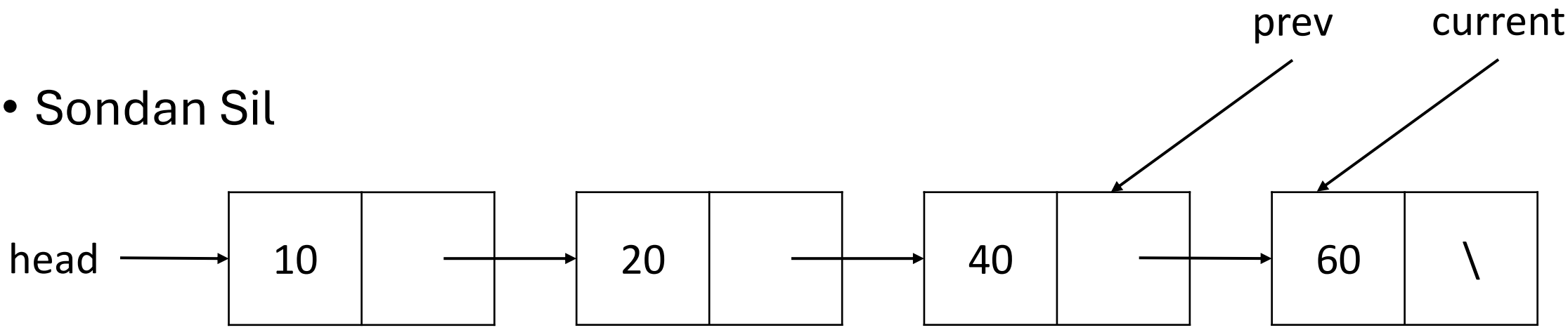
Linked List

- Sondan Sil



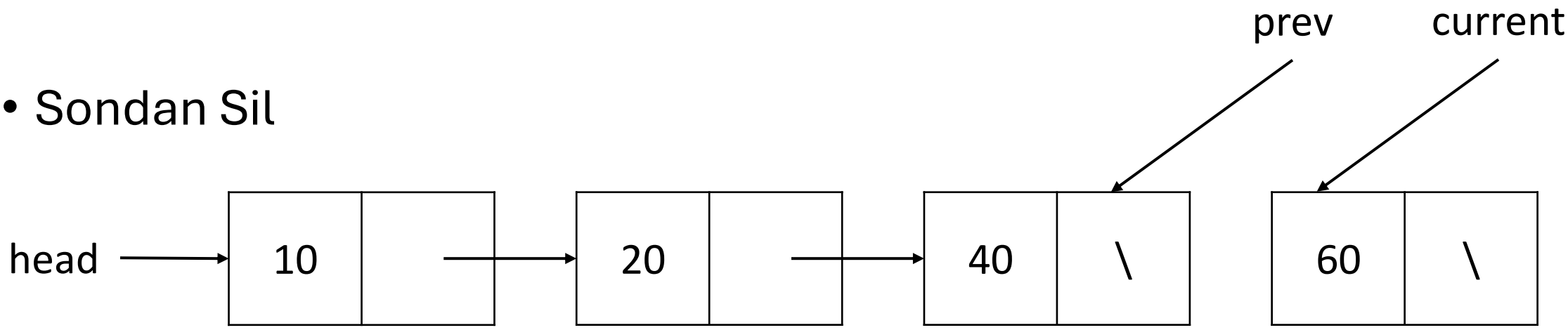
Linked List

- Sondan Sil



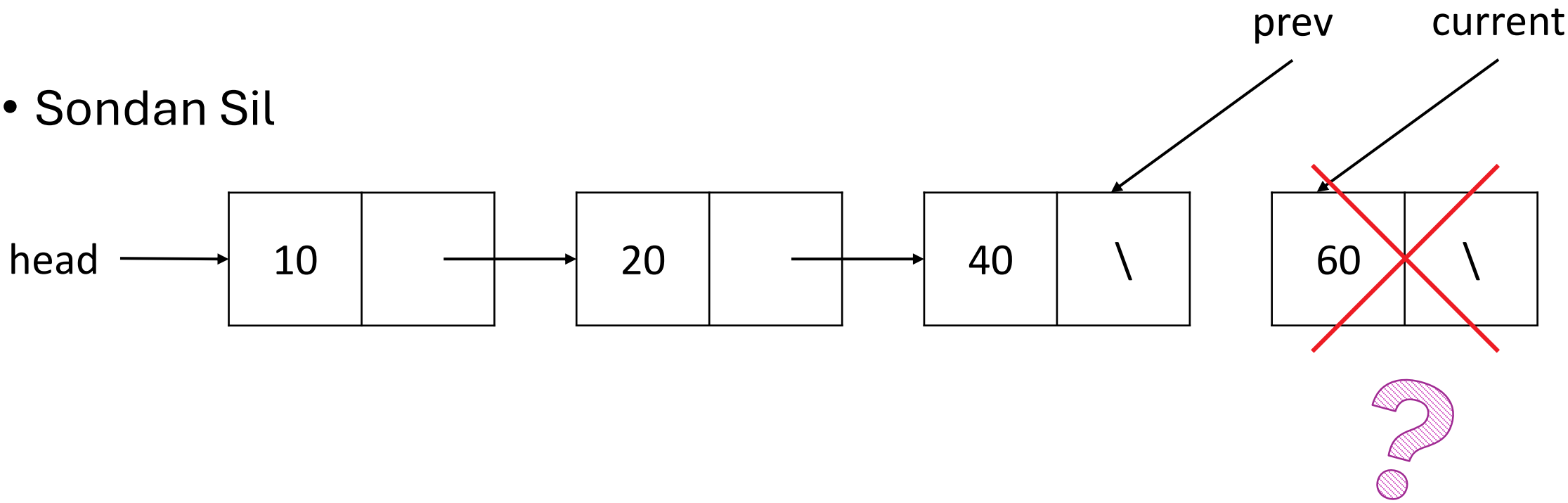
Linked List

- Sondan Sil



Linked List

- Sondan Sil



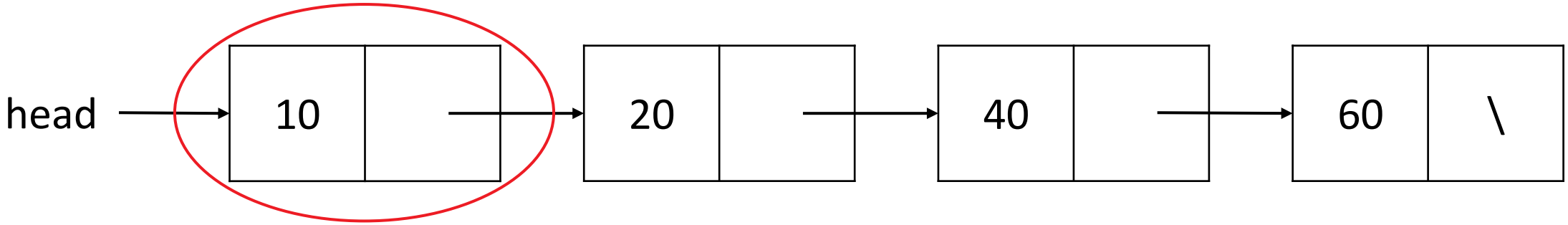
Linked List

- Sondan Sil Pseudocode

```
removeLast()
    if head != None
        if head.next == None
            head = None
        else
            current = head
            while current.next != None
                prev = current
                current = current.next
            prev.next = None
```

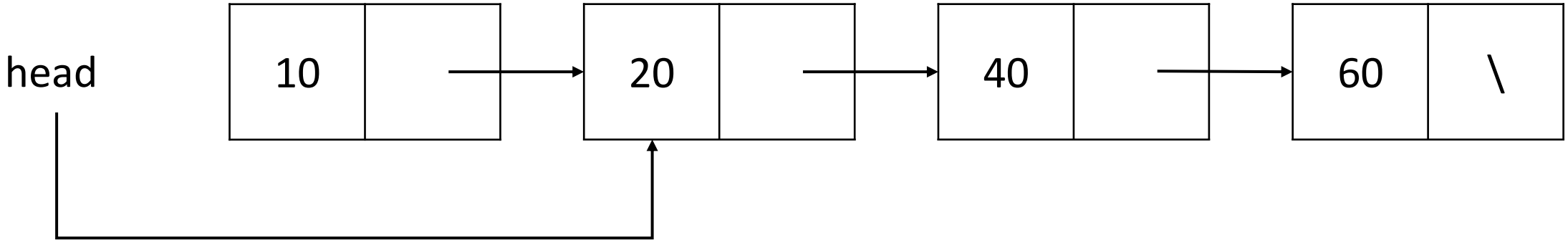
Linked List

- Baştan Sil



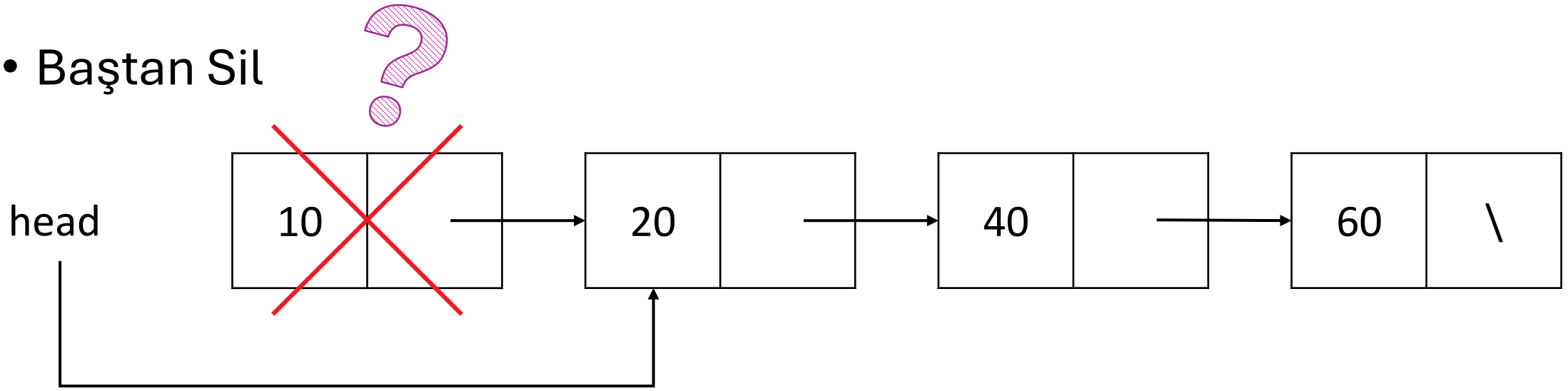
Linked List

- Baştan Sil



Linked List

- Baştan Sil



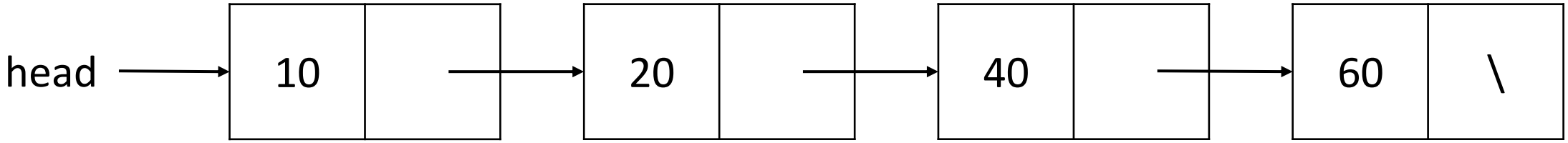
Linked List

- Baştan Sil Pseudocode

```
removeLast()
    if head != None
        head = head.next
```

Linked List

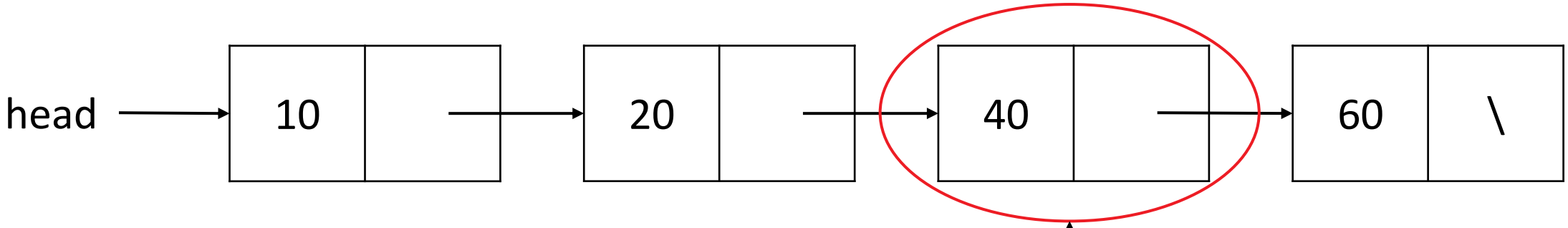
- İndeksten Sil



- İndeks = 2

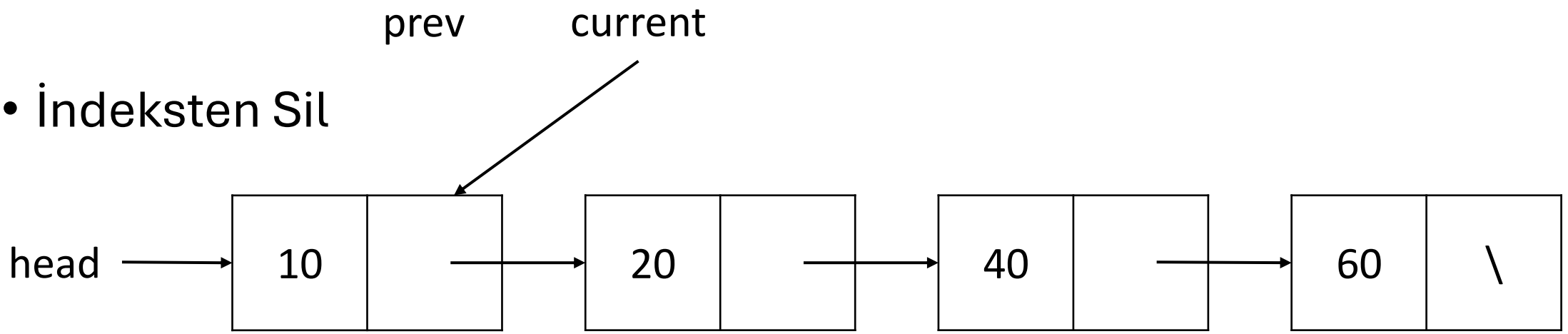
Linked List

- İndeksten Sil



- İndeks = 2

Linked List

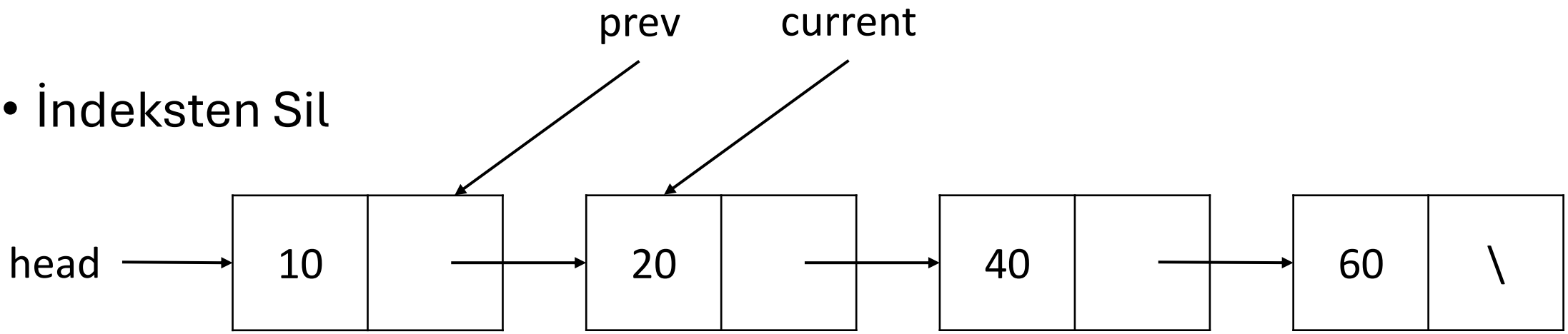


- İndeksten Sil
- İndeks = 2

sayac = 0

Linked List

- İndeksten Sil

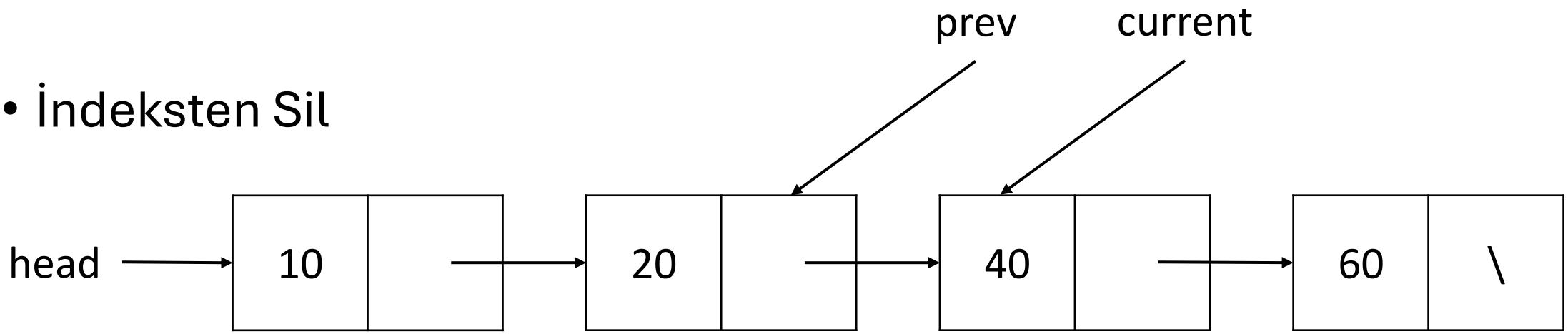


- İndeks = 2

sayac = 1

Linked List

- İndeksten Sil

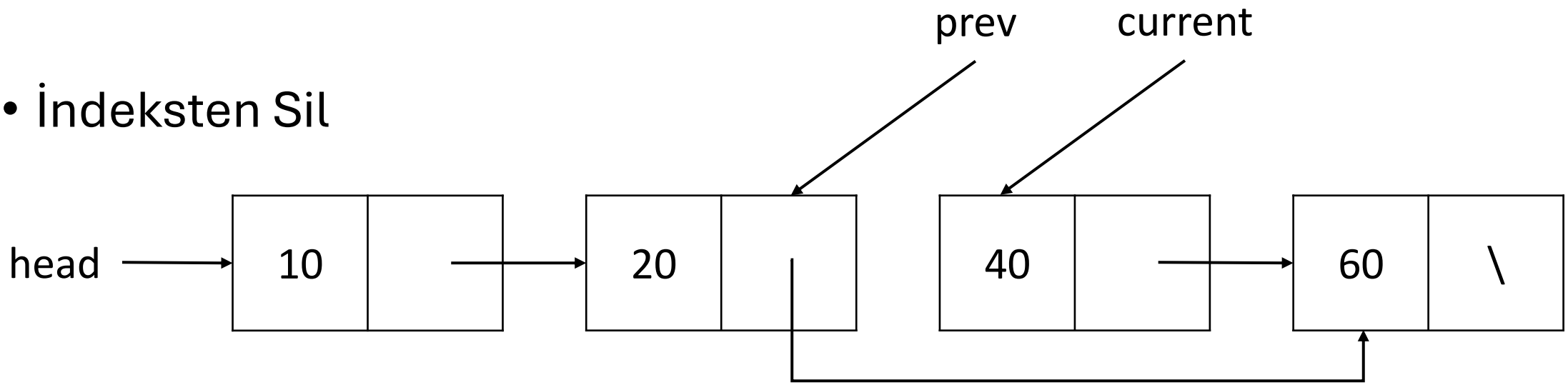


- İndeks = 2

sayac = 2

Linked List

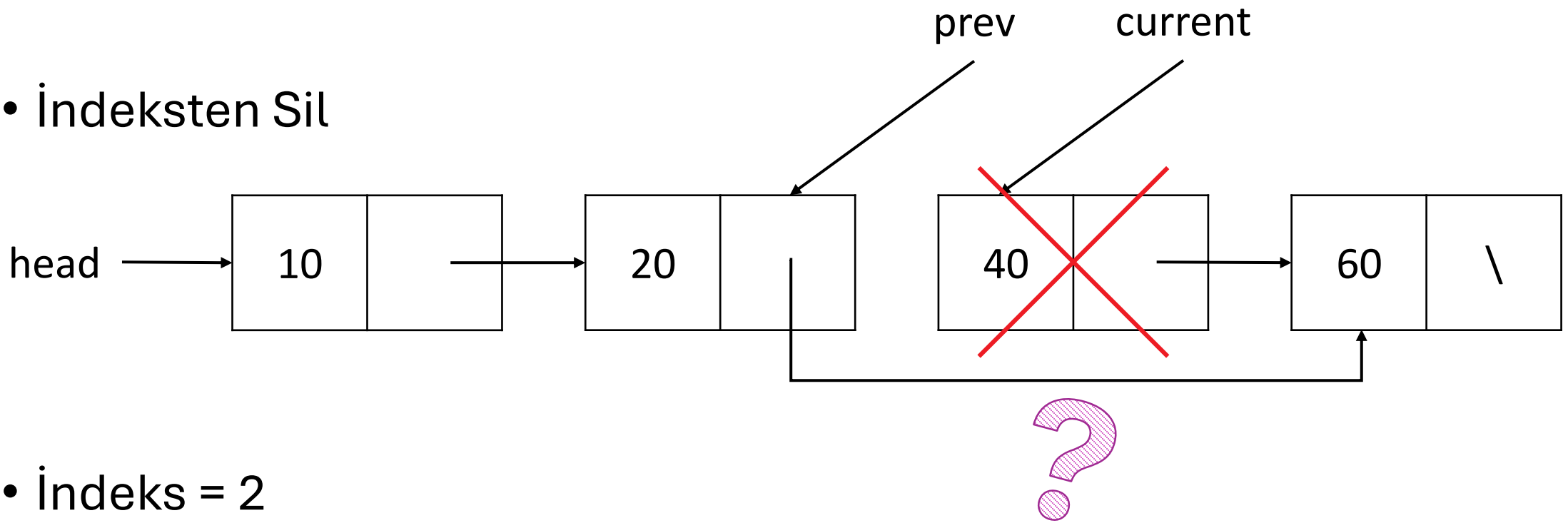
- İndeksten Sil



- İndeks = 2

Linked List

- İndeksten Sil



- İndeks = 2

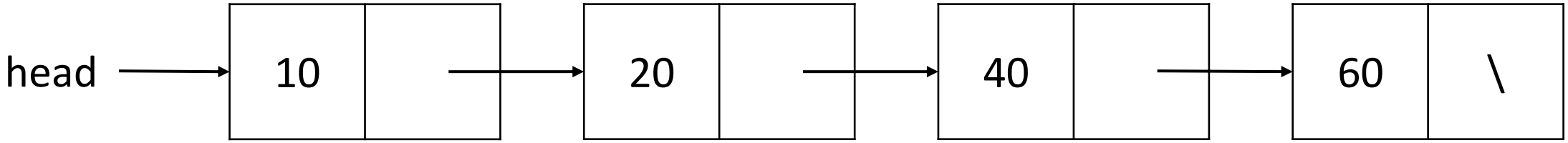
Linked List

- İndeksten Sil Pseudocode

```
removeAtIndex(idx)
    if idx <= 0
        removeFirst()
    current = head
    for idx times
        if current != None
            prev = current
            current = current.next
    if current != None
        prev.next = current.next
```

Linked List

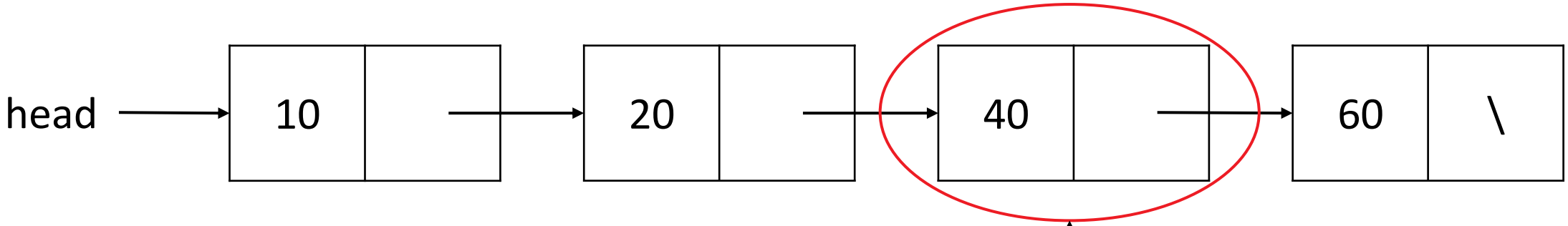
- Değere Göre Sil



- Değer = 40

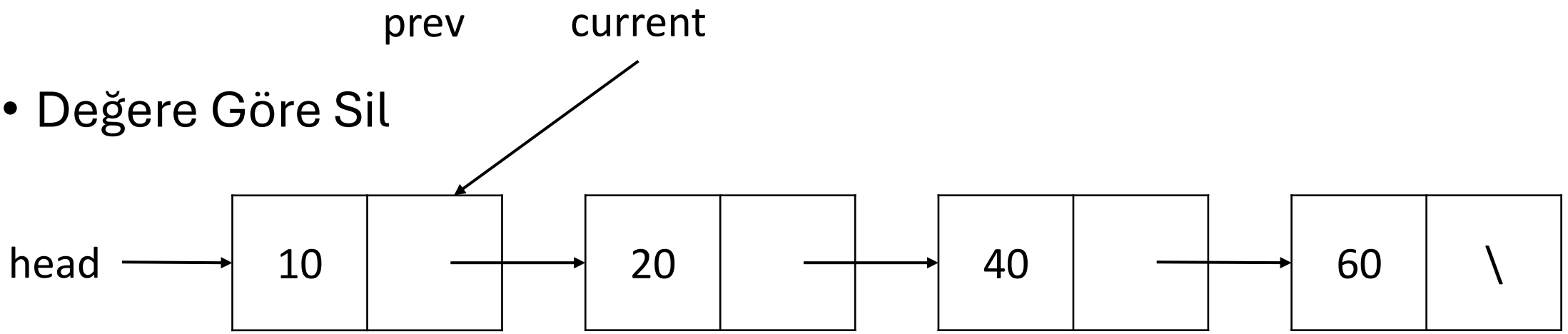
Linked List

- Değere Göre Sil



- Değer = 40

Linked List

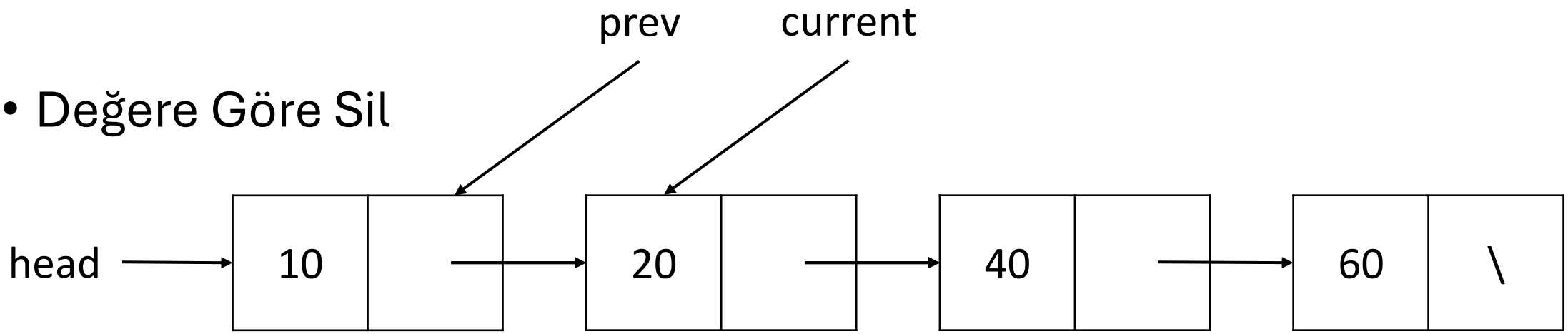


- Değere Göre Sil
- Değer = 40

10 != 40

Linked List

- Değere Göre Sil

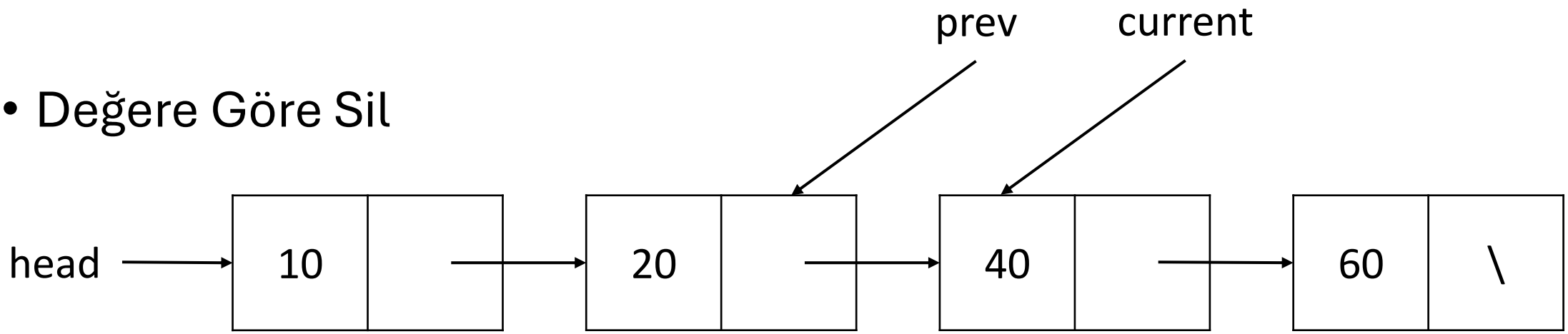


- Değer = 40

20 != 40

Linked List

- Değere Göre Sil

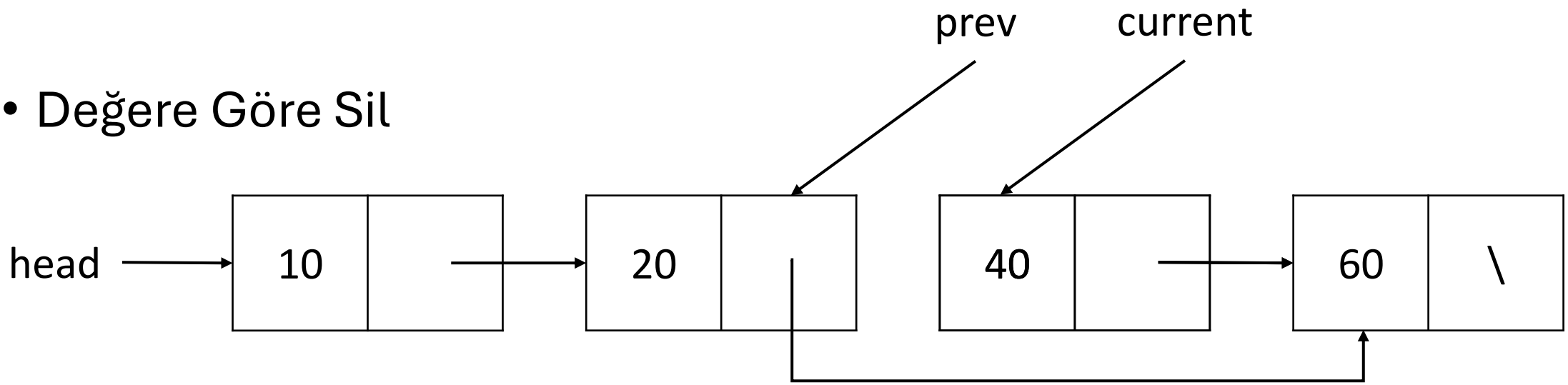


- Değer = 40

40 = 40

Linked List

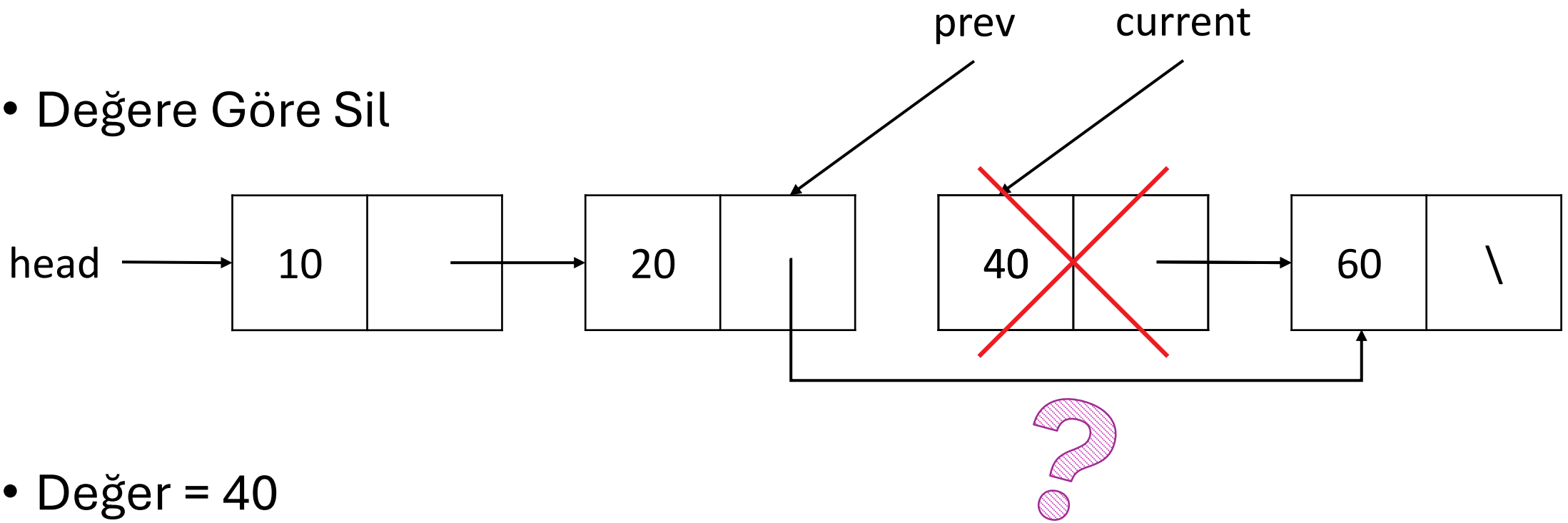
- Değere Göre Sil



- Değer = 40

Linked List

- Değere Göre Sil



- Değer = 40

Linked List

- Değere Göre Sil Pseudocode

```
removeByValue(value)
    if value == head.data
        removeFirst()
    else
        current = head
        while current != None AND value != current.data
            prev = current
            current = current.next
        if current != None
            prev.next = current.next
```

Bağlı Liste Çeşitleri

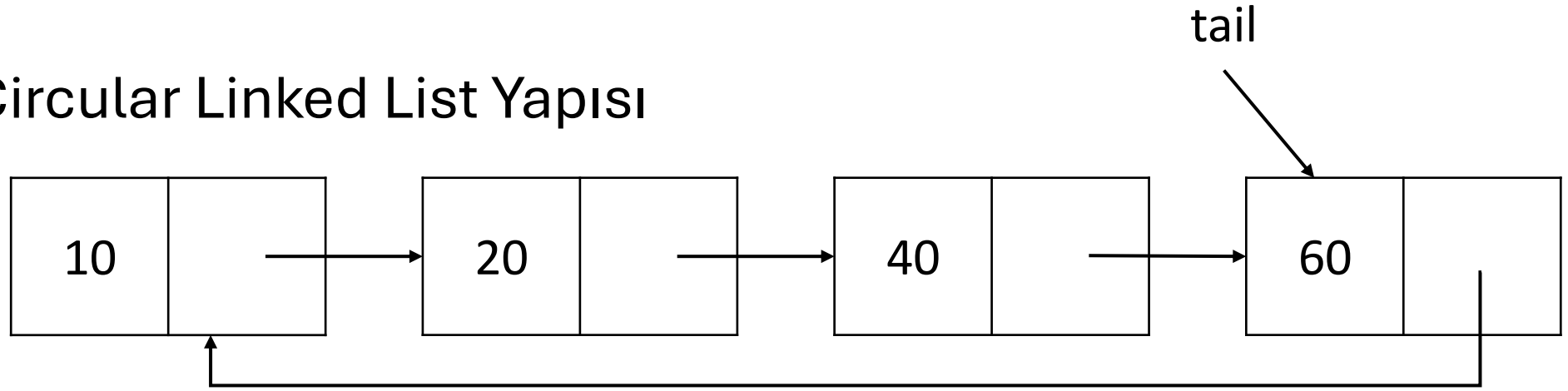
- İhtiyaç halinde bağlı listelere yeni özellikler eklenebilir
  - Genellikle işlemleri hızlandırmak ve daha verimli bir veri yapısıyla çalışmak için
- En sık kullanılan bağlı liste çeşitleri
  - Döngüsel liste (Circular linked list)
  - Çift bağlı liste (Doubly linked list)

Döngüsel Liste (Circular Linked List)

- Son node ile ilk node arasında bir bağlantı kurulur
- Son node'nun `next` alanı `None` yerine ilk node'u gösterir
- Avantajı:
  - Herhangi bir node üzerinden, liste sonuna kadar gidip, tekrar başa dönülebilir.
- Programlarken Estra Neler Yapılmalı?
  - `Node` yapısında bir değişiklik yapmaya gerek yoktur
  - Liste sonuna gelindiğini anlamak için kontrol değiştirilmeli

Circular Linked List

- Circular Linked List Yapısı



- Listeye giriş, circular linked list yapısında liste sonunu gösterir.
  - Böylece hem liste sonuna hem de liste başına tek adımda erişilebilir.
  - `tail` listenin son node'unu gösterir.
  - `tail.next` ise listenin ilk düğümünü gösterir.
  - Hem başa ekleme ve silmede hem sona eklemede kolaylık sağlanır.

Çift Bağlı Liste (Doubly Linked List)

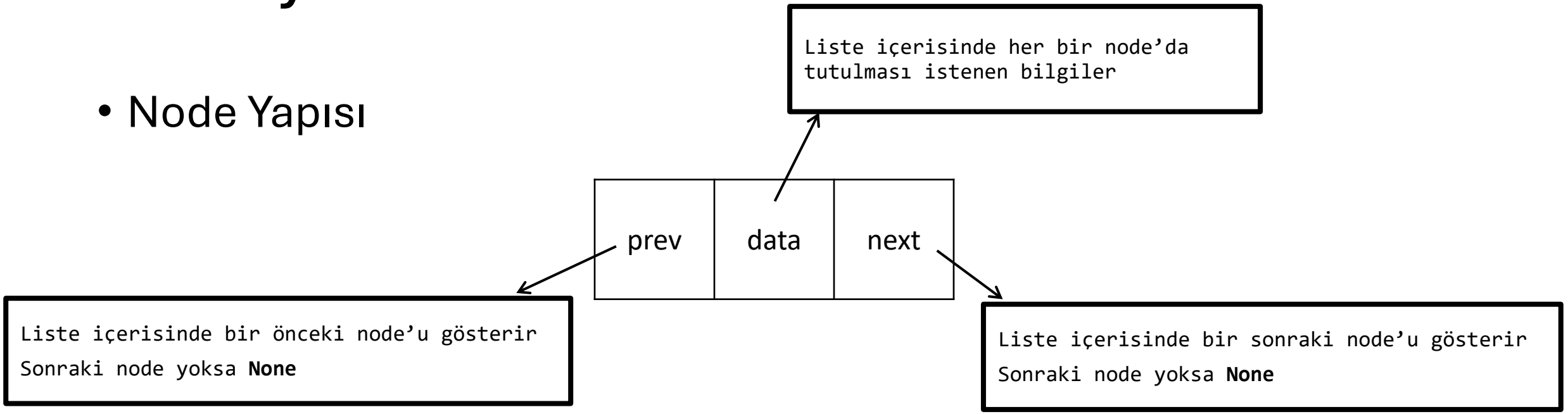
- Her bir node içinde sadece kendisinden sonra gelen değil, aynı zamanda kendisinden önce gelen node da gösterilir
  - Liste üzerinde her iki yönde de gezilebilir.
- Örnek kullanım alanları
  - Playlist ya sunum gösterimlerinde iki yönlü gezinim yapılması
  - İnternet tarayıcılarında gezilen web sayfalarının geçmişlerinin tutulması
  - Programların geri al / yenile özellikleri

Doubly Linked List

- Programlarken Estra Neler Yapılmalı?
  - Node sınıf yapısı değişecek
  - Listede hem ilk hem de son node'lar tutulacak
  - Ekleme, çıkarma gibi fonksiyonların uygulaması değişecek

Doubly Linked List

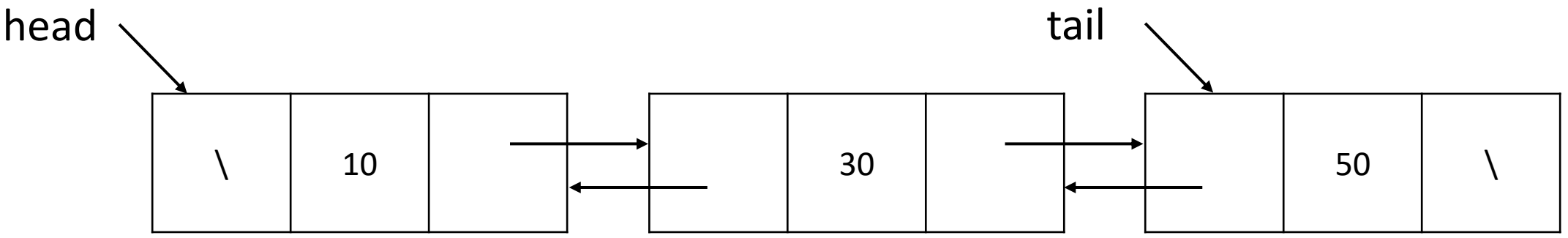
- Node Yapısı




```
class Node:
    def __init__(self, newData):
        self.data = newData
        self.prev = None
        self.next = None
```

Doubly Linked List

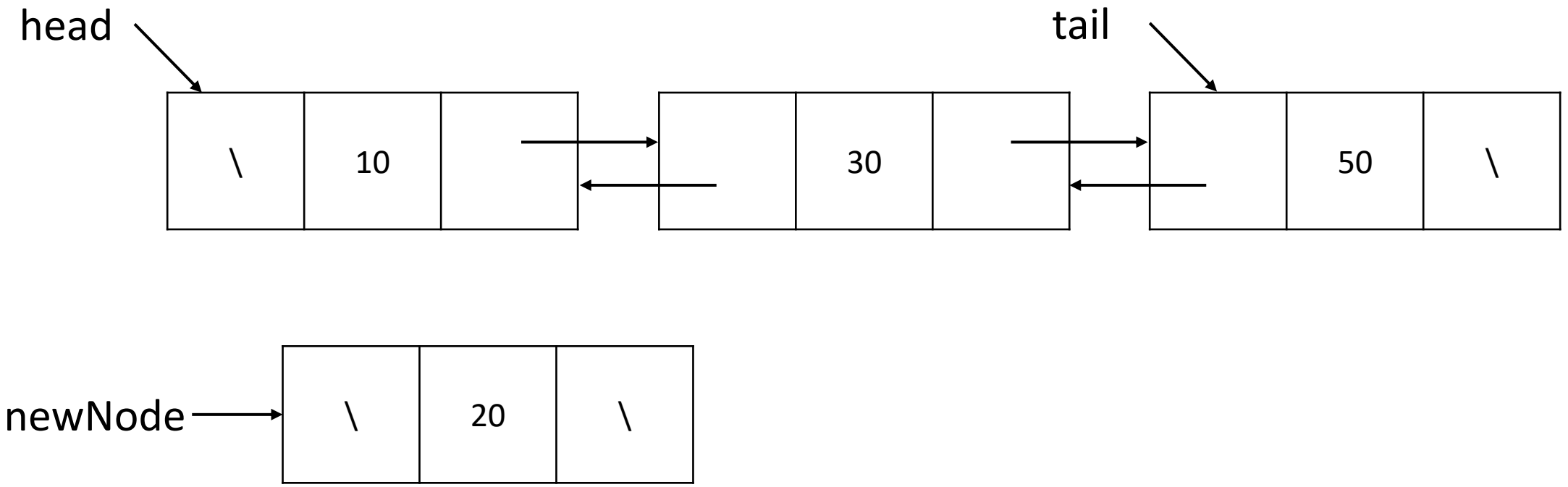
- Liste Yapısı




```
class DoublyLinkedList:
   def __init__(self):
      self.head = None
      self.tail = None
```

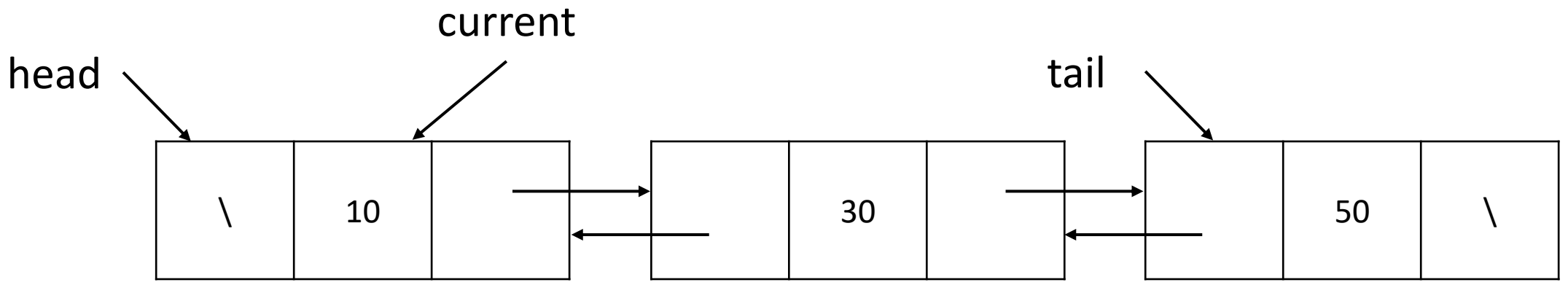
Doubly Linked List

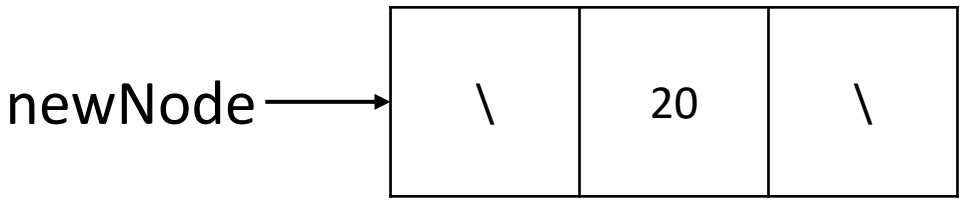
- Araya Ekle



Doubly Linked List

- Araya Ekle



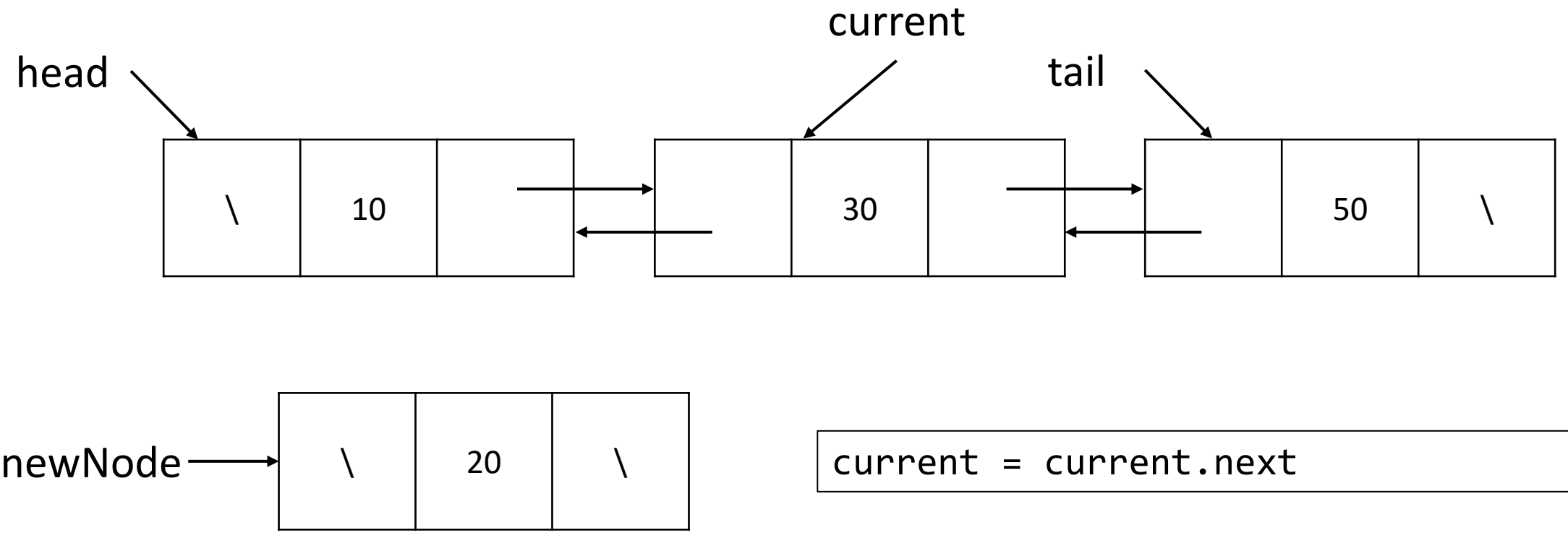



```
current = head
```

10 < 20

Doubly Linked List

- Araya Ekle

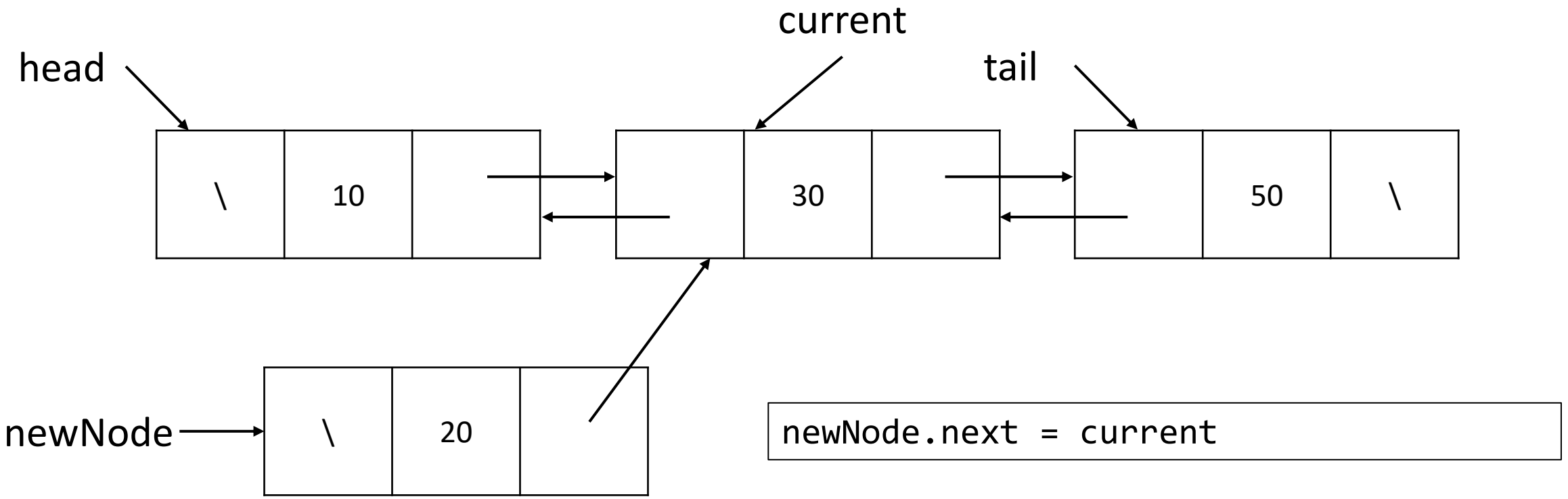



```
current = current.next
```

30 > 20

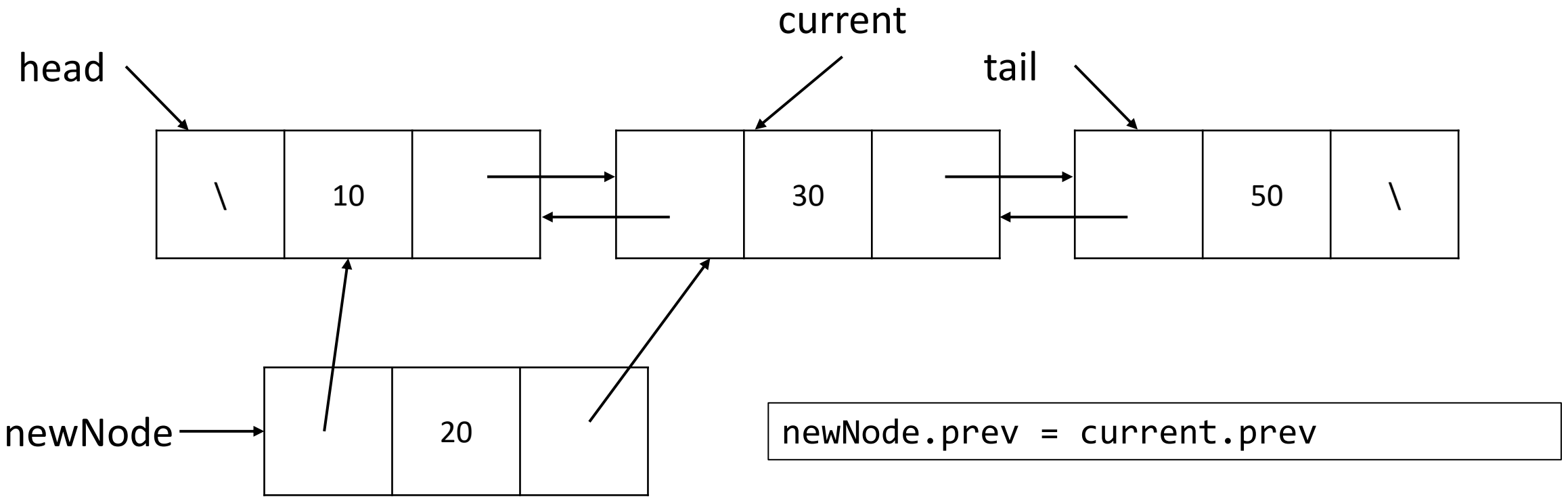
Doubly Linked List

- Araya Ekle



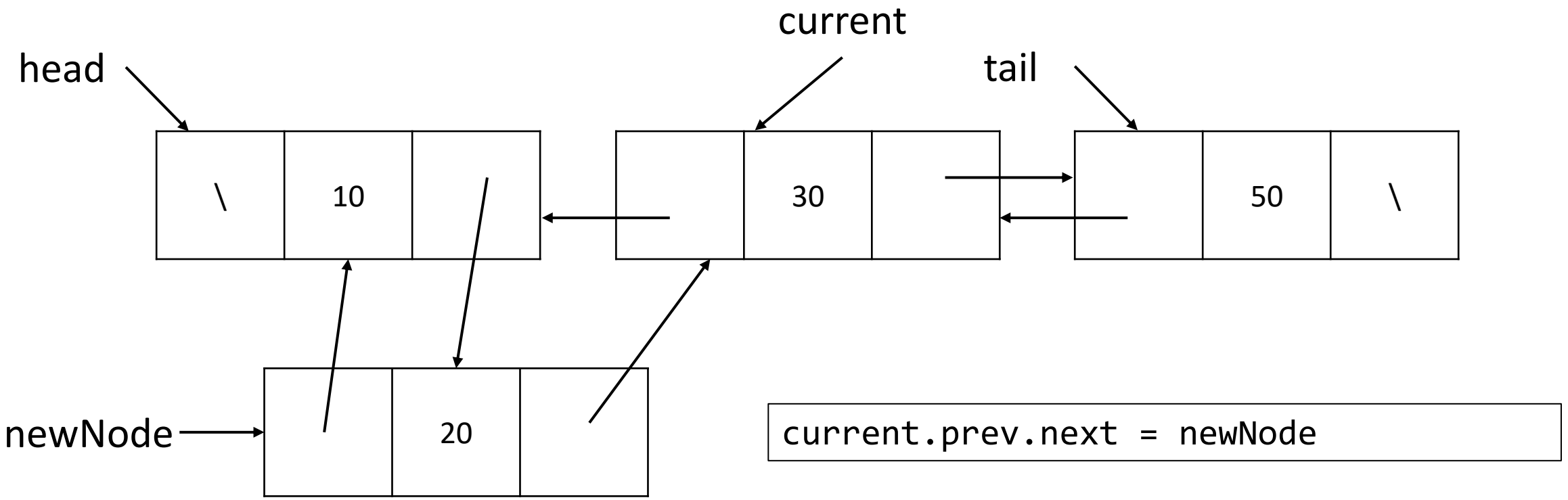
Doubly Linked List

- Araya Ekle



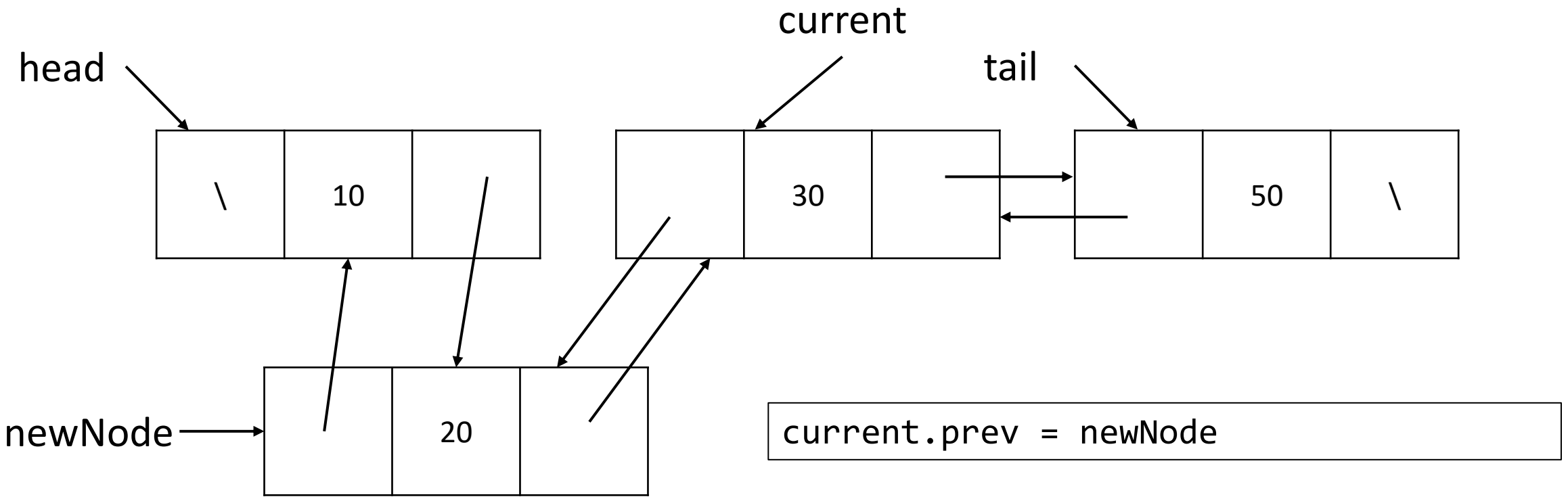
Doubly Linked List

- Araya Ekle



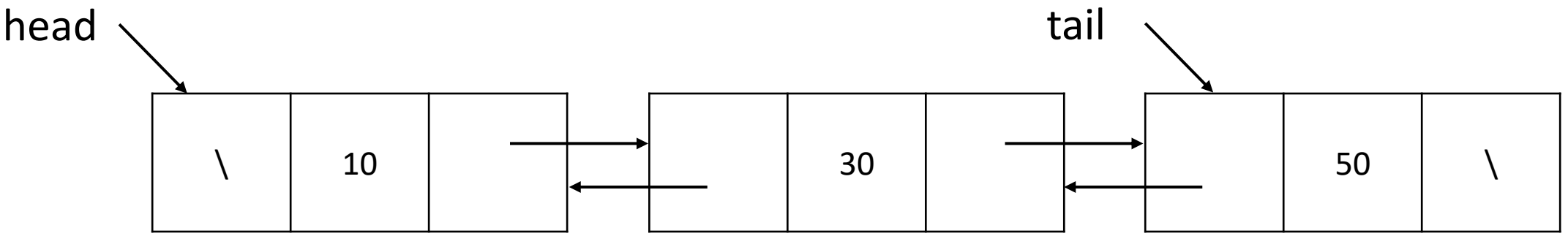
Doubly Linked List

- Araya Ekle



Doubly Linked List

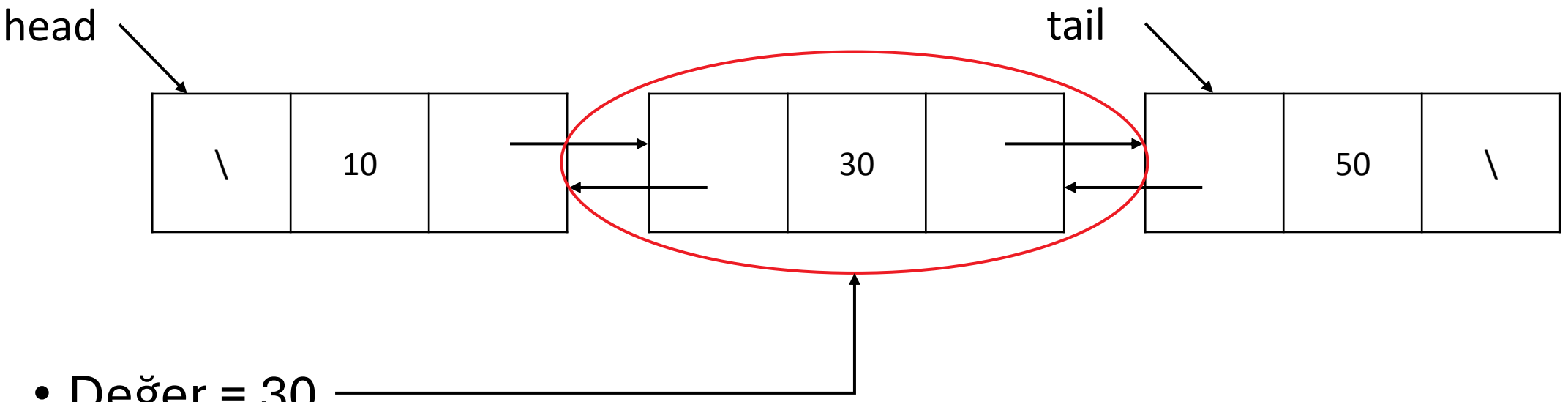
- Aradan Sil



- Değer = 30

Doubly Linked List

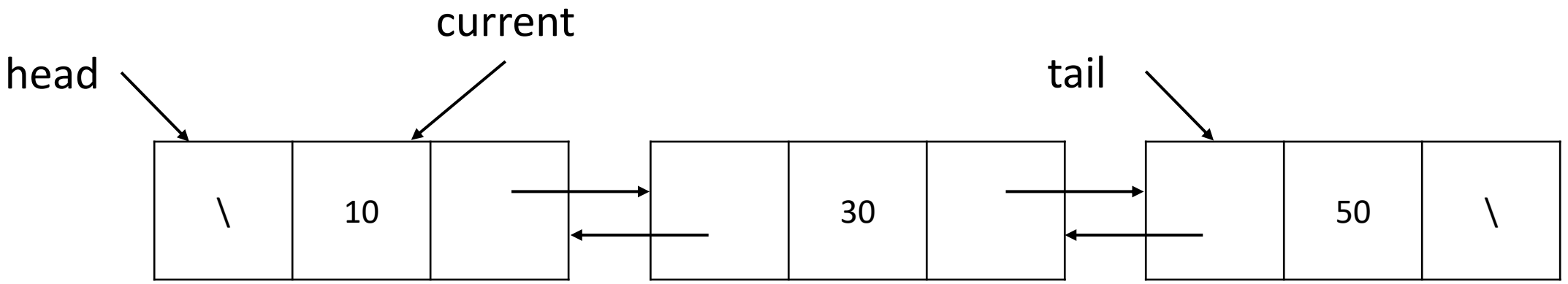
- Aradan Sil



- Değer = 30

Doubly Linked List

- Aradan Sil



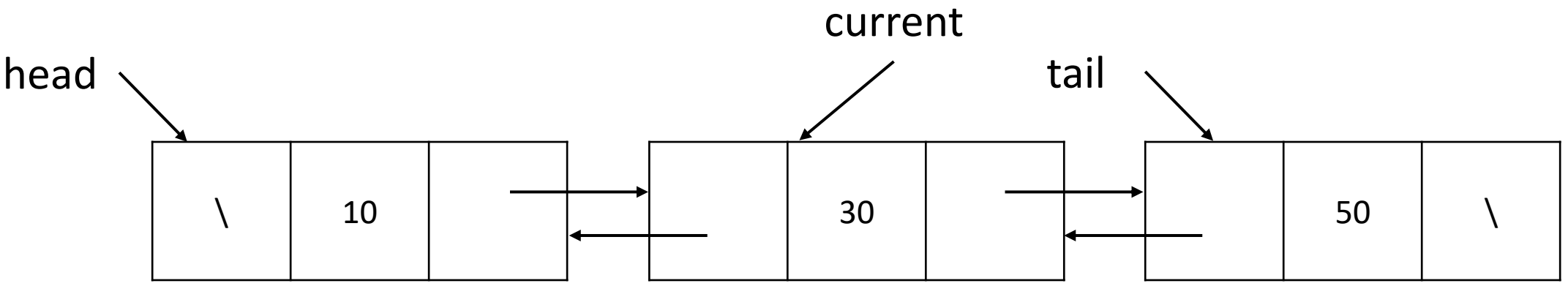
- Değer = 30

```
current = head
```

10 != 30

Doubly Linked List

- Aradan Sil



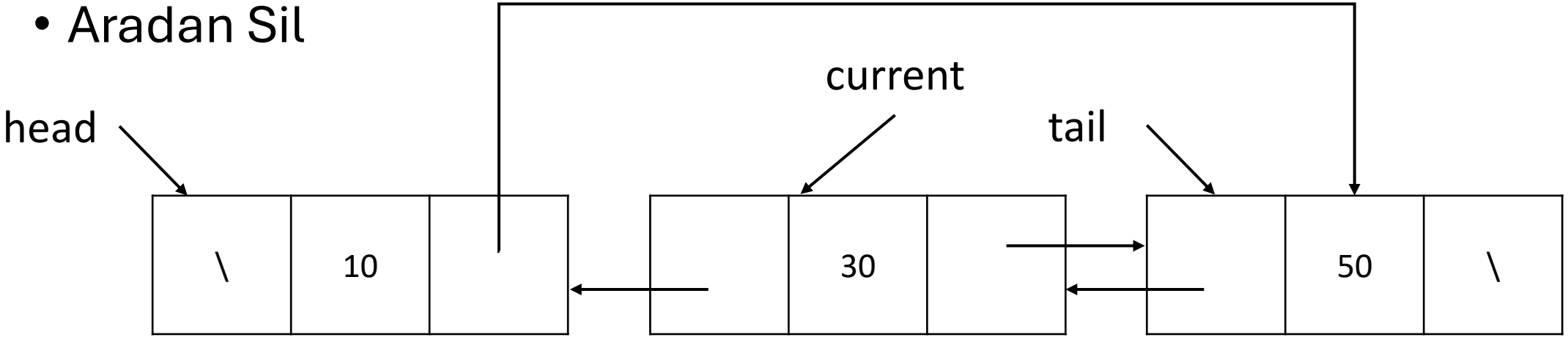
- Değer = 30

```
current = current.next
```

30 = 30

Doubly Linked List

- Aradan Sil

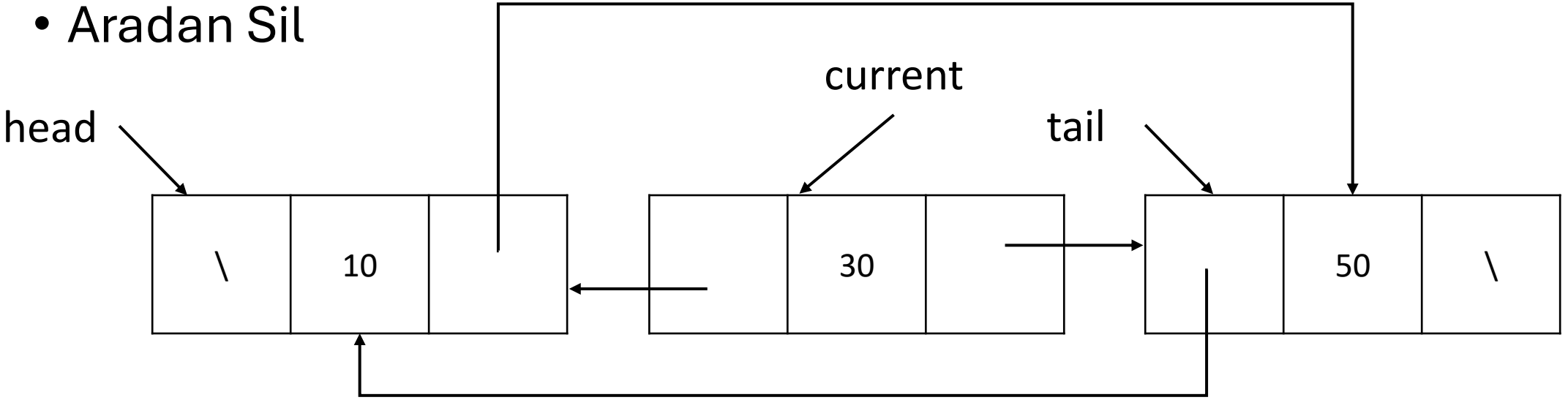


- Değer = 30

```
current.prev.next = current.next
```

Doubly Linked List

- Aradan Sil

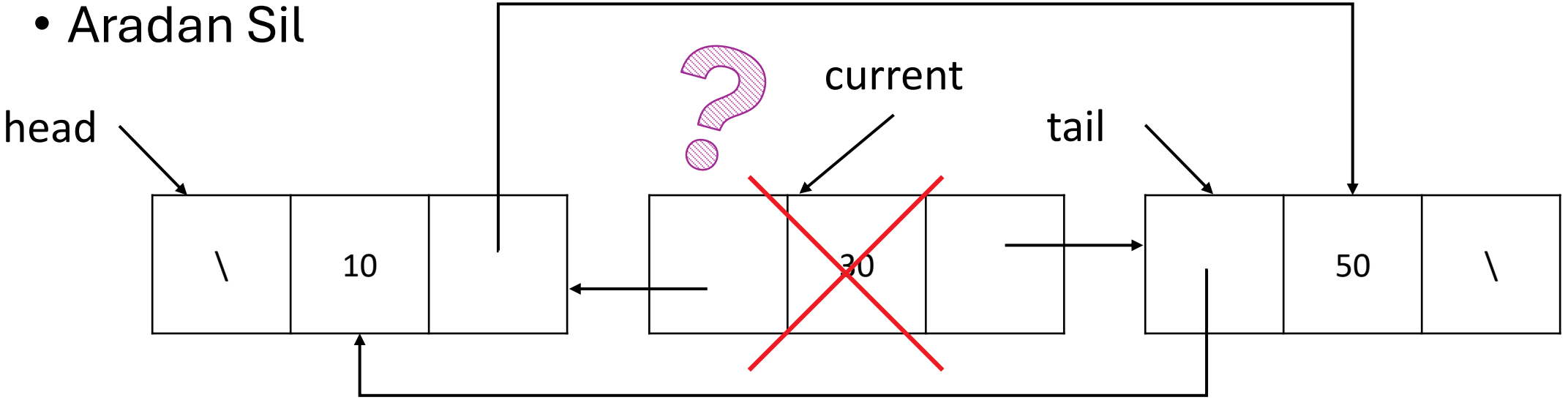


- Değer = 30

```
current.next.prev = current.prev
```

Doubly Linked List

- Aradan Sil

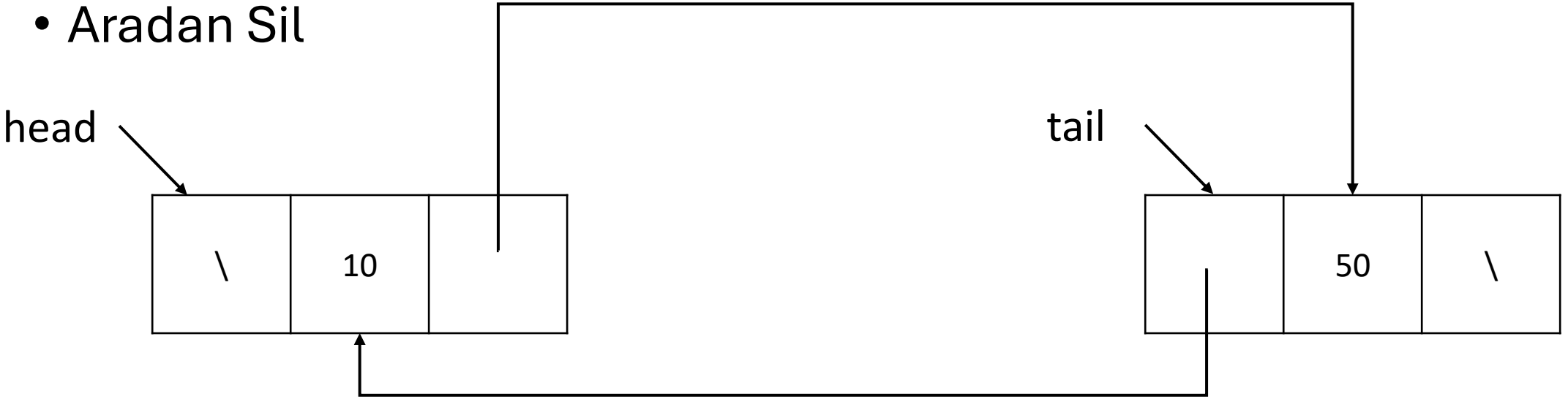


- Değer = 30

```
current = None
```

Doubly Linked List

- Aradan Sil



Referanslar

- Introduction to Algorithms, Third Edition, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, The MIT Press, 2009
- Algorithms, 4th Edition by Robert Sedgewick and Kevin Wayne, Addison-Wesley Professional, 2011
- Data Structures and Algorithms in C++, Third Edition, Adam Drozden, 2005
- https://www.geeksforgeeks.org/python/dsa/